

□> **Templates** : C:/Users/madan/Desktop/Supports de Cours
 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_01.pdf

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS 1ère année Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Décrire la structure globale du module, ses 10 séances et ses objectifs généraux
- Expliquer pourquoi l'informatique est essentielle dans la formation des enseignants de français
- Se repérer dans les modalités d'évaluation (TD, TP, QCM, projet)
- Lier chaque notion du cours aux programmes SNT Seconde et NSI Première
- Identifier les compétences numériques du CRCN / PIX attendues d'un enseignant

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** définit le cadre institutionnel des Ecoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Présentation du module : plan, objectifs, évaluation (15 min)
2. Tour de table : représentations des étudiants sur l'informatique (10 min)
3. Les 10 séances du module : timeline et graphe de dépendances (10 min)
4. Pourquoi l'informatique pour les enseignants de langues ? (20 min)
5. Liens avec les programmes SNT Seconde et NSI Première (15 min)
6. Modalités d'évaluation détaillées (10 min)

7. Questions et attentes (10 min)

1. Présentation du module

```
mindmap
  root((Informatique<br/>ENS 1ère année))
    Architecture
    Représentation info
    Binaire / Hexa
    ASCII / UTF-8
    Images bitmap
    Machine
    von Neumann
    Fetch-Decode-Execute
    Hiérarchie mémoire
    OS
    Processus & threads
    Ordonnancement
    Système de fichiers
    Algorithmique
    Fondamentaux
    Variables & types
    Conditions
    Boucles
    Complexité
     $O(1)$ ,  $O(n)$ ,  $O(n^2)$ 
    Structures
    Tableaux
    Piles & files
    Réseaux & Web
    TCP/IP & DNS
    HTTP/HTTPS
    Modèle client-serveur
    Sécurité & Éthique
    CIA triad
    Mots de passe & MFA
    RGPD & données personnelles
    Programmation Python
    Variables & types
    Conditions & boucles
    Fonctions & fichiers
    Projet pédagogique
```

Objectifs généraux du module :

- Comprendre l'architecture fondamentale d'un système informatique (CPU, mémoire, E/S, bus)
- Maîtriser les bases des systèmes d'exploitation (processus, threads, fichiers, commandes)
- Concevoir des algorithmes et évaluer leur complexité théorique ($O(1)$, $O(n)$, $O(n^2)$)
- Utiliser des structures de données classiques (tableaux, piles LIFO, files FIFO)
- Comprendre les principes réseau (IP, DNS, HTTP/HTTPS, client-serveur)
- Maîtriser les bases de la sécurité informatique (CIA, mots de passe, MFA, RGPD)
- S'initier à la programmation Python dans une perspective d'enseignement
- Relier chaque notion aux programmes SNT/NSI du secondaire

Compétences pré-requis : Baccalauréat ; notions de base en TIC (navigateur web, traitement de texte, messagerie).

Public visé : Étudiants de 1^{ère} année à l'École Normale Supérieure, Département de français, futurs enseignants de français au secondaire.

2. Les 10 séances du module

```

timeline
    title Déroulement des 10 séances
    S01[Séance 01 : Introduction au module : objectifs, plan, évaluation]
    S02[Séance 02 : Représentation de l'information : binaire, hexa, ASCII, UTF-8, images]
    S03[Séance 03 : Architecture machine : von Neumann, fetch-decode-execute, hiérarchie mémoire]
    S04[Séance 04 : Systèmes d'exploitation : processus, threads, ordonnancement, fichiers]
    S05[Séance 05 : Algorithmique fondamentale : variables, conditions, boucles, complexité]
    S06[Séance 06 : Structures de données : tableaux, piles LIFO, files FIFO, listes chaînées]
    S07[Séance 07 : Réseaux et Internet : TCP/IP, DNS, HTTP/HTTPS, client-serveur]
    S08[Séance 08 : Sécurité informatique : confidentialité, intégrité, disponibilité, RGPD]
    S09[Séance 09 : Initiation à Python : variables, conditions, boucles, fonctions, fichiers]
    S10[Séance 10 : Projet Python pédagogique : quiz, calcul de moyennes, mini-application]

graph LR
    S01[Séance 01<br/>Introduction] --> S02[Séance 02<br/>Représentation]
    S02 --> S03[Séance 03<br/>Architecture]
    S03 --> S04[Séance 04<br/>Systèmes d'exploitation]
    S04 --> S05[Séance 05<br/>Algorithmique]
    S05 --> S06[Séance 06<br/>Structures de données]
    S06 --> S07[Séance 07<br/>Réseaux]
    S07 --> S08[Séance 08<br/>Sécurité]
    S08 --> S09[Séance 09<br/>Python]
    S09 --> S10[Séance 10<br/>Projet Python]

style S01 fill:#4a90d9,color:#fff
style S05 fill:#2ecc71,color:#fff
style S10 fill:#e74c3c,color:#fff

```

Articulation pédagogique : Les séances 02 à 04 posent les fondements théoriques de l'architecture machine. Les séances 05 et 06 introduisent l'algorithmique et les structures de données, socle de la pensée computationnelle. Les séances 07 et 08 traitent des réseaux et de la sécurité, compétences citoyennes essentielles. Enfin, les séances 09 et 10 sont consacrées à la programmation Python et au projet de fin de module.

3. Importance de l'informatique pour les enseignants de langues

Constat : Le numérique est omniprésent dans l'enseignement des langues : plateformes LMS (Moodle, Google Classroom), correcteurs orthographiques (Antidote, LanguageTool), dictionnaires en ligne (WordReference, Linguee), exercices (Quizlet, LearningApps, Duolingo), classes virtuelles (Zoom, BigBlueButton), outils d'écriture collaborative (Etherpad, Framapad), et IA générative (ChatGPT pour la rédaction).

Pourquoi un enseignant de français doit-il maîtriser les bases de l'informatique ?

1. **Contribution au programme SNT Seconde :** Le programme SNT (Sciences Numériques et Technologie) est obligatoire pour tous les élèves de Seconde. Les enseignants de lettres peuvent être sollicités pour les thèmes « Données structurées », « Internet », « Réseaux sociaux » en lien avec la lecture, l'écriture et la culture informationnelle.

2. **Compétences numériques du CRCN (Cadre de Référence des Compétences Numériques) :** Le CRCN définit 5 domaines (Information et données, Communication, Création de contenus, Protection et sécurité, Environnement numérique). La certification PIX est obligatoire pour les enseignants. Un enseignant de français doit notamment maîtriser le domaine « Création

de contenus Â» (traitement de texte, prÃ©sentation multimÃ©dia, Ã©dition de documents).

3. Outils pÃ©dagogiques numÃ©riques pour la classe de franÃ§ais : Un enseignant formÃ© peut intÃ©grer des quiz interactifs (Kahoot, QuiziniÃ©re), des cartes mentales (XMind, Mindomo), des outils de dictionnaires et de correction en ligne, des plateformes d'Ã©criture (Framapad, Etherpad), des exercices de grammaire, et des ressources audio/vidÃ©o (Podcast, YouTube).

4. PensÃ©e computationnelle et analyse linguistique : La logique algorithmique (sÃ©quences, conditions, boucles) est transposable Ã l'Ã©tude de la grammaire (analyse syntaxique descendante, arbres de dÃ©rivation), Ã la phonÃ©tique (rÃgles de prononciation conditionnelles), et Ã la didactique des langues (structuration sÃ©quentielle des apprentissages).

5. Culture numÃ©rique citoyenne : Comprendre le fonctionnement d'Internet, les moteurs de recherche, les algorithmes de recommandation, la sÃ©curitÃ© des donnÃ©es personnelles et les enjeux Ã©thiques du numÃ©rique (RGPD, dÃ©sinformation, cyberharcÃ©lement) est indispensable pour former les Ã©lÃ©ves Ã une citoyennetÃ© numÃ©rique responsable.

Exemples concrets pour la classe de franÃ§ais :

- Comprendre pourquoi certains caractÃ©res franÃ§ais (Ã¢, Ã, Ã, Ã©, Ã¨) s'affichent mal dans un fichier texte Ã¢ notion d'encodage UTF-8
- Analyser le fonctionnement d'un correcteur orthographique Ã¢ algorithmes de comparaison de chaÃ¢nes
- Comprendre le rÃ©fÃ©rencement Google Ã¢ algorithme PageRank, indexation
- SÃ©curiser les donnÃ©es des Ã©lÃ©ves Ã¢ notions de chiffrement, mot de passe, RGPD

4. Liens avec les programmes SNT/NSI

ThÃ©me SNT Seconde	Lien avec le cours	Applications en franÃ§ais
DonnÃ©es structurÃ©es	SÃ©ance 02 (codage) + SÃ©ance 06 (structures)	Typologie des textes, analyse grammaticale
Internet	SÃ©ance 07 (rÃ©seaux)	Recherche documentaire, Ã©valuation des sources
Architecture machine	SÃ©ance 03 (von Neumann)	Histoire des techniques, impact sur l'Ã©criture
Algorithmes	SÃ©ance 05 (algorithmique)	Structures narratives, grammaire gÃ©nÃ©rative
RÃ©seaux sociaux	SÃ©ance 08 (sÃ©curitÃ©, Ã©thique)	Ã©ducation aux mÃ©dias, cyberharcÃ©lement

ThÃ©me NSI PremiÃ©re	Lien avec le cours
Types de donnÃ©es, opÃ©rateurs	SÃ©ance 05
Conditions et boucles	SÃ©ance 05
Tableaux, piles, files	SÃ©ance 06
ComplexitÃ© algorithmique	SÃ©ance 05
Transmission de donnÃ©es	SÃ©ance 07

5. ModalitÃ©s d'Ã©valuation

```

pie title RÃ©partition de l'Ã©valuation du module
"TD (40%)" : 40
"TP Python (35%)" : 35
  
```

"QCM (25%) " : 25

Composante	D�tail	Poids
TD	Exercices sur papier : architecture, r�seaux, algorithmique, structures	40 %
TP Python	S�ances pratiques (09 et 10) : programmation en salle machine	35 %
QCM	5 questions par s�ance (auto-�valuation formative)	25 %

R gles :

- La note de TD comprend des exercices   rendre apr s chaque s ance (sauf s ance 01)
- Le TP Python donne lieu   un projet individuel ou en bin me pr sent  en s ance 10
- Les QCM sont not s sur 5 points chacun ; seules les 5 meilleures notes comptent
- Une soutenance courte (5 minutes) du projet Python est pr vue en s ance 10
- La pr sence en cours est obligatoire (contr le continu)

QCM d'auto- valuation

1. Combien de s ances comporte le module d'informatique ?

a) 8 s ances b) 10 s ances   c) 12 s ances

2. Quel est le poids des TP Python dans l' valuation totale ?

a) 25 % b) 35 %   c) 40 %

3. Quel programme du secondaire alg rien est li    l'informatique en Seconde ?

a) NSI b) SNT   c) EMI

4. Quel est le principal objectif du module pour les futurs enseignants ?

a) Devenir programmeur professionnel b) Comprendre l'informatique pour l'enseigner   c) R parer des ordinateurs

5. Quel outil num rique est cit  comme exerciceur pour l'enseignement des langues ?

a) Photoshop b) Quizlet   c) Excel

Conclusion

Cette s ance introductive a pos  le cadre du module. L'informatique n'est pas une discipline isol e du reste de la formation : elle est au c ur des pratiques p dagogiques modernes, y compris dans l'enseignement des langues et des lettres. Les 10 s ances qui suivent sont con  ues pour donner aux futurs enseignants de fran sais les concepts fondamentaux et les comp tences pratiques n cessaires pour int grer le num rique dans leur enseignement et pour contribuer efficacement aux programmes SNT/NSI du secondaire.

Prochaine s ance : Repr sentation de l'information   binaire, hexad cimal, codage ASCII/UTF-8, codage des images bitmap.

Note p dagogique ENS

Transposition didactique : Cette séance introductive peut être adaptée pour une classe de Seconde SNT en début d'année. L'enseignant peut :

- Faire un tour de table sur les usages numériques des élèves (smartphone, réseaux sociaux, jeux vidéo)
- Présenter le programme SNT sous forme de mindmap (comme fait ici)
- Expliquer le lien entre les sciences numériques et les autres disciplines (français, histoire-géographie, mathématiques, SVT)
- Proposer une activité de réflexion : « À quoi sert un ordinateur dans la vie quotidienne ? »

Conseil pour l'enseignant formateur : Inviter les étudiants ENS à réfléchir à leur propre rapport au numérique (forces, faiblesses, attentes) pour personnaliser leur parcours d'apprentissage tout au long du module. Les étudiants peuvent tenir un journal de bord numérique de leurs apprentissages.

Prolongement : Demander aux étudiants de préparer pour la séance 2 un court texte (5 lignes) listant les appareils numériques qu'ils utilisent quotidiennement, pour amorcer la réflexion sur la représentation de l'information.

Bibliographie

- Ministère de l'Éducation nationale (2019). *Programme SNT Seconde* « Bulletin officiel spécial n°1 du 22 janvier 2019.
- Ministère de l'Éducation nationale (2019). *Programme NSI Première et Terminale* « Bulletin officiel spécial n°1 du 22 janvier 2019.
- CRCN « Cadre de Référence des Compétences Numériques (2020). <https://pix.fr/cadre-de-reference>
- Dowek, G. et al. (2019). *Sciences Numériques et Technologie* « Seconde. Éditions Hachette Éducation.
- Tanenbaum, A. & Austin, T. (2012). *Architecture de l'ordinateur* (6e éd.). Pearson (chapitres introductifs).
- Documentation Python 3 (2024). <https://docs.python.org/fr/3/>
- Académie de Versailles (2020). *Guide d'équipement numérique pour l'enseignement des lettres.*
- Fluckiger, C. (2019). *Numérique et formation des enseignants de langues.* Revue des sciences de l'Éducation.

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_02.pdf

Cours Informatique à l'ENS 1^{ère} année S^éance 02 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'1^{ère} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Convertir un nombre décimal en binaire et vice versa
- Convertir un nombre binaire en hexadécimal et vice versa
- Expliquer la différence entre ASCII et Unicode/UTF-8
- Coder un caractère à partir de son code ASCII ou Unicode
- Calculer la taille d'une image bitmap (pixels, résolution, profondeur de couleur)
- Appliquer ces notions à des exemples concrets pour l'enseignement des langues

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'égard du cadre institutionnel des Ecoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l'égard des objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** à l'égard des épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** à l'égard du cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** à l'égard de l'étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Introduction : pourquoi représenter l'information ? (5 min)
2. Le système binaire (base 2) (20 min)
3. Le système hexadécimal (base 16) (15 min)
4. Codage des caractères : ASCII, Unicode, UTF-8 (20 min)
5. Codage des images : bitmap, pixels, résolution, couleur (20 min)
6. Applications pédagogiques pour l'enseignement des langues (10 min)

1. Arbre des codages

```
graph TD
    INFO["Information numÃ©rique"] --> NOMBRES["Nombres"]
    INFO --> TEXTES["Textes"]
    INFO --> IMAGES["Images"]
    INFO --> SON["Son (Ã©chantillonnage)"]

    NOMBRES --> BIN["Binaire (base 2)<br/>0 et 1"]
    NOMBRES --> DEC["DÃ©cimal (base 10)<br/>0-9"]
    NOMBRES --> HEX["HexadÃ©cimal (base 16)<br/>0-9, A-F"]

    TEXTES --> ASCII["ASCII<br/>128 car. (7 bits)"]
    TEXTES --> UNICODE["Unicode<br/>~150 000 car."]
    UNICODE --> UTF8["UTF-8<br/>1-4 octets"]

    IMAGES --> BITMAP["Bitmap<br/>Matrice de pixels"]
    BITMAP --> COULEUR["Couleur RVB<br/>24 bits/pixel"]
    BITMAP --> NB["Noir & Blanc<br/>1 bit/pixel"]

    SON --> ECH["Ã©chantillonnage<br/>kHz + quantification"]

    style BIN fill:#e74c3c,color:#fff
    style HEX fill:#e67e22,color:#fff
    style ASCII fill:#3498db,color:#fff
    style UTF8 fill:#2ecc71,color:#fff
    style BITMAP fill:#9b59b6,color:#fff
```

L'information numÃ©rique est toujours reprÃ©sentÃ©e avec des **0** et des **1** (bits). Tout type de donnÃ©e (nombre, texte, image, son, vidÃ©o) est converti en binaire pour Ãªtre traitÃ© par un ordinateur.

2. Le systÃ©me binaire (base 2)

Principe

Le systÃ©me binaire utilise uniquement deux chiffres : **0** et **1**. Chaque position correspond Ã une puissance de 2.

2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1	

Conversion dÃ©cimal Ã binaire : mÃ©thode des divisions successives

Exemple : convertir 42 en binaire

```
42 ÷ 2 = 21, reste 0 (bit de poids faible)
21 ÷ 2 = 10, reste 1
10 ÷ 2 = 5, reste 0
5 ÷ 2 = 2, reste 1
2 ÷ 2 = 1, reste 0
1 ÷ 2 = 0, reste 1
```

Lecture des restes de bas en haut : 42₁₀ = 101010₂

Conversion binaire Ã dÃ©cimal : mÃ©thode des puissances

```
1010102 = 1×25 + 0×24 + 1×23 + 0×22 + 1×21 + 0×20
= 32 + 0 + 8 + 0 + 2 + 0
```


= 42â,â,â

```
graph LR
    subgraph "Binaire â' DÃ©cimal"
        B1["1010â,,"] --> P1["1Ã-2Ã³ + 0Ã-2Ã² + 1Ã-2Ã¹ + 0Ã-2Ã°"]
        P1 --> R1["= 8 + 0 + 2 + 0 = 10â,â,â"]
    end

    subgraph "DÃ©cimal â' Binaire"
        DEC1["13â,â,â"] --> DIV["13Ã·2=6 r1<br/>6Ã·2=3 r0<br/>3Ã·2=1 r1<br/>1Ã·2=0 r1"]
        DIV --> BIN1["= 1101â,,"]
    end

    style R1 fill:#2ecc71,color:#fff
    style BIN1 fill:#2ecc71,color:#fff
```

Bit = Binary Digit. Un octet (byte) = 8 bits. Un octet permet de coder 256 valeurs diffÃ©rentes (2â,â,â).

3. Le systÃ¨me hexadÃ©cimal (base 16)

Principe

L'hexadÃ©cimal utilise 16 chiffres : **0-9** puis **A-B-C-D-E-F** (A=10, B=11, C=12, D=13, E=14, F=15).

Pourquoi l'hexadÃ©cimal ? Parce qu'un chiffre hexa correspond exactement Ã 4 bits (un quartet), ce qui rend les conversions binaire-hexa trÃ¨s simples.

Tableau de conversion

```
block-beta
columns 4
block:group1
columns 4
B0["0000"] B1["0001"] B2["0010"] B3["0011"]
B4["0100"] B5["0101"] B6["0110"] B7["0111"]
B8["1000"] B9["1001"] BA["1010"] BB["1011"]
BC["1100"] BD["1101"] BE["1110"] BF["1111"]
end
space
block:group2
columns 4
H0["0"] H1["1"] H2["2"] H3["3"]
H4["4"] H5["5"] H6["6"] H7["7"]
H8["8"] H9["9"] HA["A"] HB["B"]
HC["C"] HD["D"] HE["E"] HF["F"]
end
space
block:group3
columns 4
D0["0"] D1["1"] D2["2"] D3["3"]
D4["4"] D5["5"] D6["6"] D7["7"]
D8["8"] D9["9"] D10["10"] D11["11"]
D12["12"] D13["13"] D14["14"] D15["15"]
end
group1 --> group2 --> group3
```

Conversion binaire â' hexa : Grouper par 4 bits Ã partir de la droite, convertir chaque groupe.

1010 1110 0101â,, â' A E 5â,â,â â' AE5â,â,â

A (1010) = 10â,â,â
 E (1110) = 14â,â,â
 5 (0101) = 5â,â,â

Conversion hexa à binaire : Chaque chiffre hexa donne 4 bits.

3Fâ,â,† â†' 0011 1111â,, â†' 111111â,,

Conversion hexa à décimal :

AE5â,â,† = Aâ-16â² + Eâ-16â¹ + 5â-16â°
 = 10â-256 + 14â-16 + 5
 = 2560 + 224 + 5
 = 2789â,â,€

Usage courant : L'hexadécimal est utilisé pour représenter les couleurs en HTML/CSS (#FF0000 pour le rouge, #FFFFFF pour le blanc, #000000 pour le noir), les adresses mémoire, et les adresses MAC.

4. Codage des caractères

ASCII (American Standard Code for Information Interchange)

- Créé en 1963, standardisé en 1967
- **7 bits** par caractère à 128 caractères (0-127)
- Contient : lettres anglaises (A-Z, a-z), chiffres (0-9), signes de ponctuation, caractères de contrôle

```
graph TD
  subgraph "Table ASCII (extrait)"
    L1["65 â†' A<br/>66 â†' B<br/>67 â†' C"]
    L2["97 â†' a<br/>98 â†' b<br/>99 â†' c"]
    L3["48 â†' 0<br/>49 â†' 1<br/>50 â†' 2"]
    L4["32 â†' espace<br/>33 â†' !<br/>63 â†' ?"]
  end

  subgraph "Limitation"
    LIM["Pas d'accents<br/>Pas de Å, Æ, Å, Åª, Å³, Å¼<br/>Pas d'arabe, chinois, etc."]
  end

  L1 --> LIM
  L2 --> LIM
  L3 --> LIM
  L4 --> LIM
```

Exemple : Le mot "Hello" en ASCII :

H â†' 72 â†' 0100 1000
 e â†' 101 â†' 0110 0101
 l â†' 108 â†' 0110 1100
 l â†' 108 â†' 0110 1100
 o â†' 111 â†' 0110 1111

Unicode et UTF-8

Unicode : Répertoire universel de caractères. Chaque caractère possède un **point de code** unique (ex: U+0041 pour 'A', U+00E9 pour 'É', U+0628 pour 'Ø').

UTF-8 : Encodage dominant sur le Web (> 95%). Utilise de **1 à 4 octets** par caractère :

- 1 octet : caractères ASCII (U+0000 à U+007F) à compatibilité ascendante
- 2 octets : lettres accentuées, alphabets grec, cyrillique, arabe (U+0080 à U+07FF)
- 3 octets : caractères asiatiques (CJK), symboles divers (U+0800 à U+FFFF)
- 4 octets : caractères rares, émojis (U+10000 à U+10FFFF)

Exemples : Å (U+0041, 1 octet 41), Å© (U+00E9, 2 octets C3 A9), Å§ (U+00E7, C3 A7), Å“ (U+0153, C5 93), Ø (U+00D8, D8 A8), ø (U+00F8, F8 80), Å (U+1F600, 4 octets F0 9F 98 80).

Application pour l'enseignement du français : Les caractères accentués (Å©, Å“, Åª, Å« , Å , Å¢, Å§, Å“, Å¡, Å®, Å¯, Å´, Å» , Å¼) sont codés sur 2 octets en UTF-8. Un fichier texte contenant du français doit être enregistré en UTF-8 pour éviter les problèmes d'affichage (apparition de Å« ?? Å» ou de caractères illisibles).

5. Codage des images bitmap

Schéma d'une image bitmap

```
graph TD
    subgraph "Image bitmap 4-4"
        P11["(255,0,0)"]
        P12["(255,0,0)"]
        P13["(0,0,255)"]
        P14["(0,0,255)"]
        P21["(255,0,0)"]
        P22["(255,0,0)"]
        P23["(0,0,255)"]
        P24["(0,0,255)"]
        P31["(0,255,0)"]
        P32["(0,255,0)"]
        P33["(255,255,0)"]
        P34["(255,255,0)"]
        P41["(0,255,0)"]
        P42["(0,255,0)"]
        P43["(255,255,0)"]
        P44["(255,255,0)"]
    end

    subgraph "Légende"
        R["Rouge = (255,0,0)"]
        B["Bleu = (0,0,255)"]
        G["Vert = (0,255,0)"]
        J["Jaune = (255,255,0)"]
    end

    style R fill:#ff0000,color:#fff
    style B fill:#0000ff,color:#fff
    style G fill:#00ff00,color:#000
    style J fill:#ffff00,color:#000
```

Pixel = Picture Element. Plus petite unité d'une image numérique.

Résolution : nombre de pixels (largeur — hauteur). Exemple : 1920 Å— 1080 = 2 073 600 pixels (Full HD).

Profondeur de couleur : nombre de bits par pixel (bpp).

- 1 bpp : noir et blanc (2 couleurs)
- 8 bpp : 256 couleurs (palette)
- 24 bpp : 16,7 millions de couleurs (RVB : 8 bits par canal)
- 32 bpp : 24 bits + canal alpha (transparence)

Calcul de la taille d'une image

Taille (en octets) = Largeur Å— Hauteur Å— (Profondeur / 8)

Exemple : image 1920 Å— 1080 en 24 bits
 Taille = 1920 Å— 1080 Å— 3
 = 6 220 800 octets
 Å^ 5,93 Mo

Exercice : Une image de 800 Å— 600 pixels en noir et blanc (1 bit/pixel) pÅse :

800 Å— 600 Å— 1/8 = 60 000 octets Å^ 58,6 Ko

Comparaison : Une photo en 24 bits de 12 mÅgapixels (4000 Å— 3000) pÅse :

4000 Å— 3000 Å— 3 = 36 000 000 octets = 36 Mo (non compressé)

C'est pourquoi les formats compressés (JPEG, PNG) sont utilisés.

QCM d'auto-évaluation

1. Combien de bits y a-t-il dans un octet ?

a) 4 bits b) 8 bits c) 16 bits

2. Quelle est la valeur décimale de 1010₂ ?

a) 8 b) 10 c) 12

3. Quel encodage de caractères est dominant sur le Web ?

a) ASCII b) UTF-8 c) ISO-8859-1

4. Combien d'octets utilise UTF-8 pour coder le caractère « Å » ?

a) 1 octet b) 2 octets c) 3 octets

5. Une image de 1024 × 768 pixels en 24 bits pèse approximativement :

a) 768 Ko b) 2,25 Mo c) 5 Mo

Conclusion

La représentation de l'information est le fondement de toute l'informatique. Les conversions binaire/hexa/décimal, le codage des caractères (ASCII, Unicode/UTF-8) et le codage des images sont des notions essentielles pour comprendre comment un ordinateur stocke et manipule les données. Pour un enseignant de français, la maîtrise du codage UTF-8 est particulièrement importante pour éviter les problèmes d'affichage des caractères accentués dans les documents numériques.

Prochaine séance : Architecture machine et le modèle de von Neumann et le cycle d'exécution.

Note pédagogique ENS

Transposition didactique : En classe de SNT Seconde (thème « Données structurées »), on peut proposer :

- Activité de débrouille : coder son prénom en binaire avec des cartes noires (0) et blanches (1)
- Activité : convertir des mots français en ASCII, puis constater que les accents ne sont pas codés
- Activité : créer une image bitmap en noir et blanc sur papier quadrillé (chaque case = un pixel)

Lien avec les langues : Expliquer pourquoi certains caractères français (À, Á, Â, Ã) peuvent poser problème dans des fichiers exportés depuis Word vers un autre logiciel (problème d'encodage). Montrer l'importance de toujours enregistrer en UTF-8.

Conseil pour l'enseignant formateur : Pour aider les élèves à visualiser les concepts abstraits de codage, privilégier les activités de débrouille (cartes, papier quadrillé) avant de passer aux outils numériques. Utiliser le site « Codage binaire » pour des exercices interactifs. Rappeler que la maîtrise du codage des caractères est essentielle pour les enseignants de français qui manipulent des fichiers texte avec accents.

Prolongement : Les étudiants peuvent approfondir en explorant le codage des images (bitmap, RVB, compression JPEG) et le codage audio (échantillonnage, quantification). Proposer une activité complémentaire : créer une image bitmap en Python à partir d'une matrice de pixels en niveaux de

gris (projet interdisciplinaire mathématiques-informatique).

Bibliographie

- Goupille, P. (2014). *Technologie des ordinateurs*. Éditions Dunod (chapitres 1 et 2).
- Tanenbaum, A. & Austin, T. (2012). *Architecture de l'ordinateur* (6e éd.). Pearson.
- Unicode Consortium (2024). *The Unicode Standard* » <https://unicode.org>
- Dowek, G. et al. (2019). *SNT Seconde* » *Sciences Numériques et Technologie*. Hachette.
- Wikipedia » « Codage des caractères », « Image bitmap », « UTF-8 » (consultation libre).

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_03.pdf

Cours Informatique à l'ENS 1^{ère} année S^éance 03 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'1^{ère} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Décrire le modèle de von Neumann et ses 5 unités fonctionnelles
- Expliquer le déroulement du cycle fetch-decode-execute
- Distinguer les différents niveaux de la hiérarchie mémoire (registres, cache, RAM, SSD, HDD)
- Calculer un temps d'accès moyen et comparer les performances
- Relier ces notions aux programmes SNT (architecture) et NSI (performances)

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à définir le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à fixer les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** à déterminer les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** à cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** à étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Introduction : qu'est-ce qu'un ordinateur ? (5 min)
2. Le modèle de von Neumann à les 5 unités (20 min)
3. Le cycle fetch-decode-execute (20 min)
4. La hiérarchie mémoire à pyramide des mémoires (20 min)
5. Comparaison des performances et calculs pratiques (15 min)
6. Applications pédagogiques pour SNT/NSI (10 min)

1. Le modèle de von Neumann

```

graph TD
    UC["Unité de Contrôle<br/>(Control Unit)"] --> UAL["UAL<br/>ALU"]
    UC --> MEM["Mémoire<br/>(RAM)"]
    UC --> E_S["Entrées / Sorties<br/>(I/O)"]

    REG["Registres"] --> UAL
    REG --> UC

    MEM --> BUS["Bus de données<br/>Bus d'adresses<br/>Bus de contrôle"]
    BUS --> UC
    BUS --> UAL
    BUS --> E_S

    subgraph "Les 5 unités fonctionnelles"
        U1["1. Unité de Contrôle"]
        U2["2. UAL (Arithmétique & Logique)"]
        U3["3. Mémoire (données + programme)"]
        U4["4. Entrées"]
        U5["5. Sorties"]
    end

    style UC fill:#e74c3c,color:#fff
    style UAL fill:#3498db,color:#fff
    style MEM fill:#2ecc71,color:#fff
    style E_S fill:#f39c12,color:#fff
    style REG fill:#9b59b6,color:#fff
    style BUS fill:#95a5a6,color:#fff

```

Le modèle de von Neumann (proposé par John von Neumann en 1945) est l'architecture fondamentale de presque tous les ordinateurs modernes. Ses 5 unités sont :

1.1 Unité de Contrôle (Control Unit)

- Dirige et coordonne toutes les actions de l'ordinateur
- Lit les instructions en mémoire et les interprète
- Génère les signaux de contrôle pour les autres unités
- Contient le **compteur ordinal** (PC = Program Counter) qui pointe vers la prochaine instruction
- Contient le **registre d'instruction** (IR = Instruction Register) qui stocke l'instruction en cours

1.2 Unité Arithmétique et Logique (UAL / ALU)

- Effectue les opérations arithmétiques : addition, soustraction, multiplication, division
- Effectue les opérations logiques : ET, OU, NON, XOR
- Effectue les comparaisons : égalité, supérieur, inférieur
- Utilise des **registres** (accumulateurs) pour stocker les opérations et les résultats

1.3 Mémoire (unité de stockage)

- Stocke **à la fois** les données et les programmes (= principe de von Neumann)
- Organisée en cellules adressables, chaque cellule contenant un mot mémoire (8, 16, 32 ou 64 bits)
- Deux opérations : **lecture** (read) et **écriture** (write)

1.4 Entrées / Sorties (I/O)

- **Entrées** : clavier, souris, microphone, appareil photo, scanner, réseau

- **Sorties** : Écran, haut-parleurs, imprimante, Réseau
- Communication via les **bus** (données, adresses, contrôle)

1.5 Les bus

- **Bus de données** : transporte les données entre les unités (bidirectionnel)
- **Bus d'adresses** : transporte l'adresse mémoire à lire/écrire (unidirectionnel CPU à mémoire)
- **Bus de contrôle** : transporte les signaux de synchronisation (lecture/écriture, horloge)

Largeur des bus : Un bus d'adresses de 32 bits peut adresser $2^{32} = 4$ Go de mémoire. Un bus de 64 bits peut adresser $2^{64} = 16$ Eo (exaoctets).

2. Le cycle fetch-decode-execute

```
sequenceDiagram
    participant PC as Compteur<br/>Ordinal (PC)
    participant MEM as Mémoire (RAM)
    participant IR as Registre<br/>Instruction
    participant UC as Unité de<br/>Contrôle
    participant UAL as UAL (ALU)

    Note over PC,UAL: Cycle d'exécution d'une instruction

    UC->>MEM: 1. FETCH : Envoyer (PC) sur bus d'adresses
    MEM-->>UC: Lire instruction à l'adresse PC
    UC->>IR: Stocker instruction dans IR
    UC->>PC: PC ← PC + 1 (incrémement)

    UC->>IR: 2. DECODE : Analyser l'opcode (ex: ADD)
    UC->>UC: Identifier les opérandes nécessaires

    UC->>MEM: 3. FETCH Operandes : Lire opérandes
    MEM-->>UAL: Transmettre opérandes à l'UAL

    UAL->>UAL: 4. EXECUTE : Effectuer l'opération
    UAL-->>MEM: 5. STORE : Écrire le résultat en mémoire
```

Détail des étapes

Étape 1 : Fetch (recherche) :

- L'unité de contrôle place le contenu du **PC** (compteur ordinal) sur le bus d'adresses
- La mémoire renvoie l'instruction située à cette adresse
- L'instruction est chargée dans le **IR** (registre d'instruction)
- Le PC est incrémenté pour pointer vers l'instruction suivante

Étape 2 : Decode (décodage) :

- L'unité de contrôle décode l'**opcode** (code opératoire) pour déterminer l'opération à effectuer
- Elle identifie les **opérandes** (registres ou adresses mémoire concernés)
- Elle active les circuits appropriés dans l'UAL

Étape 3 : Execute (exécution) :

- L'UAL effectue l'opération (addition, soustraction, comparaison, etc.)

- Le résultat est temporairement stocké dans un registre accumulateur

Étape 4 : Store (rangement) :

- Le résultat est écrit dans un registre ou en mémoire (RAM)
- Le cycle recommence avec l'instruction suivante (pointé par PC)

```
graph TD
    DEBUT("Début") --> FETCH["FETCH<br/>Charger instruction<br/>depuis RAM vers IR"]
    FETCH --> DECODE["DECODE<br/>Analyser l'opcode<br/>dans l'unité de contrôle"]
    DECODE --> EXECUTE["EXECUTE<br/>Effectuer l'opération<br/>dans l'UAL"]
    EXECUTE --> STORE["STORE<br/>Écrire le résultat<br/>en mémoire/registre"]
    STORE --> FETCH
```

```
style DEBUT fill:#2ecc71,color:#fff
style FETCH fill:#3498db,color:#fff
style DECODE fill:#e67e22,color:#fff
style EXECUTE fill:#e74c3c,color:#fff
style STORE fill:#9b59b6,color:#fff
```

Fréquence d'horloge : Chaque cycle est cadencé par l'horloge du processeur. Un CPU à 3 GHz effectue 3×10^9 cycles par seconde. Une instruction simple (ADD) peut prendre 1 cycle, une instruction complexe (DIV) peut en prendre plusieurs.

3. La hiérarchie mémoire

Pyramide des mémoires

```
graph TB
    subgraph "Hiérarchie mémoire (vitesse d'accroissante, capacité croissante)"
        R1["Registres<br/>~ 1 Ko<br/>~ 0,3 ns<br/>< 1 cycle"]
        R2["Cache L1<br/>~ 32-64 Ko<br/>~ 1 ns<br/>3-4 cycles"]
        R3["Cache L2<br/>~ 256-512 Ko<br/>~ 3 ns<br/>~ 10 cycles"]
        R4["Cache L3<br/>~ 8-16 Mo<br/>~ 10 ns<br/>~ 30 cycles"]
        R5["RAM (DDR4/5)<br/>~ 8-64 Go<br/>~ 50-100 ns<br/>~ 150 cycles"]
        R6["SSD (NVMe)<br/>~ 256 Go - 2 To<br/>~ 10 000 ns<br/>~ 30 000 cycles"]
        R7["HDD<br/>~ 500 Go - 10 To<br/>~ 10 000 000 ns<br/>~ 30 M cycles"]
    end
```

```
R1 --> R2 --> R3 --> R4 --> R5 --> R6 --> R7
```

```
style R1 fill:#e74c3c,color:#fff
style R2 fill:#e67e22,color:#fff
style R3 fill:#f1c40f,color:#000
style R4 fill:#2ecc71,color:#fff
style R5 fill:#3498db,color:#fff
style R6 fill:#9b59b6,color:#fff
style R7 fill:#34495e,color:#fff
```

Les différentes mémoires

1. Registres :

- Intégrés dans le processeur
- Capacité : quelques dizaines à quelques centaines d'octets
- Temps d'accès : ~ 0,3 ns (1 cycle d'horloge)
- Utilisation : stockage des opérations et résultats immédiats de l'UAL

2. Mémoire cache (L1, L2, L3) :

- Mémoire rapide et coûteuse, proche du CPU
- Stocke les données et instructions fréquemment utilisées (principe de **localité spatiale et temporelle**)
- L1 : la plus rapide, dédiée par cœur
- L2 : partagée ou par cœur, moins rapide
- L3 : partagée entre tous les cœurs

3. RAM (Random Access Memory) :

- Mémoire volatile (perdue à l'extinction)
- Capacité : 8 à 64 Go (selon la configuration)
- Temps d'accès : ~ 50 ns (DDR4)
- Organisation : barrettes DIMM/SO-DIMM

4. SSD (Solid State Drive) :

- Mémoire non volatile (Flash NAND)
- Capacité : 256 Go à 2 To
- Temps d'accès : ~ 10 000 ns (10 µs)
- Avantages : rapide, silencieux, résistant aux chocs
- Inconvénient : cycles d'écriture limités

5. HDD (Hard Disk Drive) :

- Mémoire magnétique non volatile
- Capacité : 500 Go à 10+ To
- Temps d'accès : ~ 10 000 000 ns (10 ms)
- Avantage : coût par Go très faible
- Inconvénient : lent, mécanique, bruyant

Principe de localité

- **Localité temporelle** : une donnée accédée récemment sera probablement réutilisée bientôt (boucles)
- **Localité spatiale** : une donnée proche d'une donnée accédée sera probablement accédée bientôt (tableaux)

Ces principes expliquent l'efficacité du cache : le CPU copie automatiquement en cache les données fréquemment utilisées, ce qui accélère considérablement l'exécution.

Calcul de temps d'accès moyen

Exemple : taux de hit cache = 95 %, temps cache = 1 ns, temps RAM = 50 ns.

$$\text{Temps moyen} = 0,95 \times 1 \text{ ns} + 0,05 \times 50 \text{ ns} = 0,95 + 2,5 = 3,45 \text{ ns}$$

Sans cache, le temps d'accès serait de 50 ns. Avec un cache à 95 % de hits, on gagne un facteur 14.

QCM d'auto-évaluation

1. Combien d'unités fonctionnelles composent l'architecture de von Neumann ?

a) 3 unités b) 5 unités c) 7 unités

2. Que signifie l'acronyme UAL ?

a) Unité Arithmétique et Logique c) Unité d'Application Linéaire b) Unité d'Accès Local

3. Dans le cycle fetch-decode-execute, que se passe-t-il à l'étape FETCH ?

a) L'instruction est exécutée par l'UAL b) L'instruction est chargée depuis la RAM c) Le résultat est stocké en mémoire

4. Quelle est la mémoire la plus rapide dans la hiérarchie ?

a) Cache L3 b) RAM c) Registres

5. Un SSD est environ combien de fois plus rapide qu'un HDD ?

a) 2 fois b) 100 fois c) 10 000 fois

Conclusion

L'architecture de von Neumann est le modèle universel de tous les ordinateurs modernes. Les 5 unités (contrôle, UAL, mémoire, entrées, sorties) fonctionnent en synergie via le cycle fetch-decode-execute. La hiérarchie mémoire (registres → cache → RAM → SSD → HDD) est conçue pour concilier vitesse et capacité : plus on s'éloigne du CPU, plus la mémoire est lente et volumineuse. Ces notions sont fondamentales pour comprendre les performances d'un système informatique.

Prochaine séance : Systèmes d'exploitation → processus, threads, ordonnancement et fichiers.

Note pédagogique ENS

Transposition didactique : Pour enseigner l'architecture en SNT Seconde (thème « Architecture machine ») :

- Activité d'branchage simuler le cycle fetch-decode-execute avec des élèves jouant les rôles (unité de contrôle, mémoire, UAL)
- Construire une pyramide physique de la mémoire avec des gobelets ou des boîtes pour visualiser la hiérarchie
- Comparer les temps d'accès avec des analogies : registre = votre cerveau (1 sec), cache = bureau (3 sec), RAM = bibliothèque (5 min), SSD = ville voisine (2h), HDD = pays lointain (semaines)

Lien avec les langues : Faire le parallèle entre la mémoire cache (stockage temporaire des données fréquentes) et le vocabulaire actif/passif dans l'apprentissage des langues.

Conseil pour l'enseignant formateur : Veiller à ne pas noyer les étudiants dans les détails techniques de l'architecture. L'objectif est qu'ils comprennent le cheminement d'une instruction (du clic à l'affichage) et non les spécificités des microprocesseurs. Utiliser des vidéos animées du cycle d'exécution (ex: « How a CPU works » de la chaîne YouTube CrashCourse) pour renforcer la compréhension visuelle.

Prolongement : Les étudiants peuvent explorer l'assembleur MIPS ou RISC-V via le simulateur en ligne « MARS » pour visualiser l'exécution instruction par instruction. Proposer une activité complémentaire : comparer les architectures CISC (Intel) et RISC (ARM) et leurs implications dans les appareils mobiles utilisés en contexte scolaire (tablettes, smartphones).

Bibliographie

- von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Contrat W-670-ORD-4926.
- Tanenbaum, A. & Austin, T. (2012). *Architecture de l'ordinateur* (6e éd.). Pearson.
- Hennessy, J. & Patterson, D. (2017). *Computer Architecture: A Quantitative Approach* (6e éd.). Morgan Kaufmann.
- Goupille, P. (2014). *Technologie des ordinateurs*. Dunod.
- Dowek, G. et al. (2019). *SNT Seconde*. Hachette (chapitre « Architecture machine »).

Templates : C:/Users/madan/Desktop/Supports de Cours
 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
 250626/Template/Presentation_BELACEL_V3.pptx

PDF: Cours_Informatique_ENS_1ère_année_FR_04.pdf

Cours Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 04 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques spÃ©cifiques

Ã€ l'issue de cette sÃ©ance, l'Ã©tudiant est capable de :

- Expliquer le rÃ´le d'un systÃ©me d'exploitation et la distinction noyau/espace utilisateur
- DÃ©finir un processus et le distinguer d'un thread
- DÃ©crire les 3 Ã©tats d'un processus (ready, running, waiting)
- Expliquer le principe de l'ordonnancement CPU (FIFO, Round Robin)
- Naviguer dans une arborescence de fichiers et comprendre les permissions Unix
- Adapter ces connaissances pour des activitÃ©s SNT et des situations d'enseignement

Contexte rÃ©glementaire algÃ©rien

Ce cours s'inscrit dans le cadre rÃ©glementaire algÃ©rien de formation des enseignants :

- **ArrÃªtÃ© ministÃ©riel portant crÃ©ation des ENS (cycle Licence/Master)** â€” dÃ©finit le cadre institutionnel des Ã©coles Normales SupÃ©rieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algÃ©riennes** â€” fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **ArrÃªtÃ© 712 fixant le programme des concours ENS** â€” dÃ©termine les Ã©preuves et les programmes des concours d'accÃ©s aux ENS, en cohÃ©rence avec les contenus enseignÃ©s.
- **RÃ©fÃ©rentiel des compÃ©tences numÃ©riques algÃ©rien** â€” cadre national (en cours d'Ã©laboration) dÃ©finissant les compÃ©tences numÃ©riques attendues des enseignants.
- **Benchmark des 10 universitÃ©s algÃ©riennes** â€” Ã©tude comparative des pratiques d'enseignement de l'informatique, consultÃ©e pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Introduction : qu'est-ce qu'un systÃ©me d'exploitation ? (5 min)
2. Architecture de l'OS : noyau, espace noyau, espace utilisateur (15 min)
3. Processus et threads : dÃ©finitions, Ã©tats, cycles de vie (20 min)
4. Ordonnancement CPU : FIFO, Round Robin, prioritÃ©s (20 min)
5. SystÃ©me de fichiers : arborescence, rÃ©pertoires, permissions (20 min)
6. Applications pÃ©dagogiques et ligne de commande (10 min)

1. R  le du syst  me d'exploitation

```

graph TB
    USER["Utilisateur<br/>(applications)"] --> UI["Interface Utilisateur<br/>(GUI / CLI)"]
    UI --> OS["Syst  me d'exploitation"]
    OS --> KERNEL["Noyau (Kernel)"]

    subgraph "Services du noyau"
        PROC["Gestion des processus<br/>Ordonnancement"]
        MEM["Gestion m  moire<br/>M  moire virtuelle"]
        FS["Syst  me de fichiers<br/>Stockage & permissions"]
        DEV["Gestion p  riph  riques<br/>Pilotes (drivers)"]
        NET["R  seau<br/>TCP/IP stack"]
    end

    KERNEL --> PROC
    KERNEL --> MEM
    KERNEL --> FS
    KERNEL --> DEV
    KERNEL --> NET

    PROC --> HW["Mat  riel<br/>(CPU, RAM, disques, ...)"]
    MEM --> HW
    FS --> HW
    DEV --> HW
    NET --> HW

    style OS fill:#e74c3c,color:#fff
    style KERNEL fill:#3498db,color:#fff
    style HW fill:#95a5a6,color:#fff

```

D  finition : Un syst  me d'exploitation (OS) est un logiciel qui fait l'interface entre le mat  riel et les applications. Il g  re les ressources de l'ordinateur et fournit des services aux programmes.

R  les principaux :

1. **Abstraction mat  rielle :** cacher la complexit   du mat  riel aux applications
2. **Multiplexage des ressources :** partager le CPU, la m  moire, les disques entre plusieurs programmes
3. **Isolation et protection :** emp  cher un programme d'acc  der aux donn  es d'un autre
4. **Gestion des entr  es/sorties :** communiquer avec les p  riph  riques via des pilotes (drivers)

Exemples d'OS : Windows 10/11 (Microsoft), Linux (Ubuntu, Debian, Fedora), macOS (Apple), Android (Google), iOS (Apple).

Noyau (Kernel) : c  ur de l'OS, charg   de la gestion des ressources. Il s'ex  cute en **mode noyau** (acc  s total au mat  riel). Les applications s'ex  cutent en **mode utilisateur** (acc  s restreint). Les appels syst  me (syscalls) permettent de passer du mode utilisateur au mode noyau.

2. Processus et threads

2.1 D  finition d'un processus

Processus : un programme en cours d'ex  cution, avec son propre espace m  moire (code, donn  es, pile, tas), ses fichiers ouverts, et son contexte d'ex  cution.

Chaque processus poss  de :

- Un **PID** (Process IDentifier) â€” numÃ©ro unique
- Un **Ã©tat** (ready, running, waiting)
- Un **compteur ordinal** (PC) â€” prochaine instruction Ã exÃ©cuter
- Des **registres** (contexte CPU)
- Une **table des fichiers ouverts**
- Un **espace mÃ©moire** privÃ©

2.2 Threads

Thread (fil d'exÃ©cution) : unitÃ© d'exÃ©cution plus lÃ©gÃ¨re qu'un processus. Un processus peut contenir plusieurs threads qui partagent le mÃªme espace mÃ©moire.

DiffÃ©rence processus vs thread :

CaractÃ©ristique	Processus	Thread
Espace mÃ©moire	IndÃ©pendant (isolÃ©)	PartagÃ© avec les threads du mÃªme processus
CrÃ©ation	Plus lourde (copie des ressources)	Plus lÃ©gÃ¨re
Communication	IPC (pipes, sockets, signaux)	MÃ©moire partagÃ©e directe
Isolation	Forte (un crash n'affecte pas les autres)	Faible (un crash peut tout dÃ©truire)
Commutation	Plus coÃ»teuse	Plus rapide

2.3 Les 3 Ã©tats d'un processus

```
stateDiagram-v2
    [*] --> NOUVEAU : CrÃ©ation (fork)
    NOUVEAU --> READY : PrÃªt (chargÃ© en mÃ©moire)
    READY --> RUNNING : Ã‰lu par<br/>l'ordonnanceur
    RUNNING --> READY : Interruption<br/>(temps Ã©coulÃ©)
    RUNNING --> WAITING : Attente d'une<br/>ressource (E/S)
    WAITING --> READY : Ressource<br/>disponible
    RUNNING --> TERMINE : Fin du processus (exit)
    TERMINE --> [*]
```

note right of READY : En attente de CPU
 note right of RUNNING : En cours d'exÃ©cution
 note right of WAITING : BloquÃ© (I/O, sleep)

Transitions d'Ã©tats :

- **Nouveau Ã©t' PrÃªt (Ready)** : le processus est crÃ©Ã© et placÃ© dans la file des prÃªts
- **PrÃªt Ã©t' Ã‰lu (Running)** : l'ordonnanceur sÃ©lectionne un processus prÃªt et lui alloue le CPU
- **Ã‰lu Ã©t' PrÃªt** : le quantum de temps est Ã©coulÃ©, le processus est remis dans la file des prÃªts
- **Ã‰lu Ã©t' BloquÃ© (Waiting)** : le processus demande une opÃ©ration d'E/S (lecture disque, attente rÃ©seau)
- **BloquÃ© Ã©t' PrÃªt** : l'opÃ©ration d'E/S est terminÃ©e, le processus retourne dans la file des prÃªts
- **Ã‰lu Ã©t' TerminÃ©** : le processus a fini son exÃ©cution (appel Ã `exit()`)

Changement de contexte (context switch) : Lorsque le CPU passe d'un processus Ã un autre, il doit sauvegarder le contexte du processus sortant (registres, PC) et restaurer celui du processus entrant. Cette opÃ©ration a un coÃ»t (quelques microsecondes).

3. Ordonnancement CPU

L'**ordonnanceur (scheduler)** décide quel processus prêt obtient le CPU. Objectifs : équité, efficacité, temps de réponse minimal.

3.1 FIFO (First In, First Out)

```
graph LR
    subgraph "File des processus prêts (FIFO)"
        P1["P1<br/>Arrivé à t=0"] --> P2["P2<br/>Arrivé à t=2"]
        P2 --> P3["P3<br/>Arrivé à t=4"]
        P3 --> P4["P4<br/>Arrivé à t=6"]
    end

    subgraph "Exécution sur le CPU"
        EXEC["P1 (0-5) | P2 (5-8) | P3 (8-10) | P4 (10-12)"]
    end

    P1 --> EXEC
```

Principe : Le premier processus arrivé est le premier servi. Pas de préemption (non préemptif). Une fois qu'un processus a le CPU, il le garde jusqu'à ce qu'il se termine ou se bloque.

Avantages : Simple à implémenter, équitable pour les processus longs.

Inconvénients : Temps d'attente moyen élevé pour les processus courts (effet de convoi).

3.2 Round Robin (RR)

```
gantt
    title Ordonnancement Round Robin (quantum = 2 unités)
    dateFormat X
    axisFormat %s

    section P1 (durée 6)
        Tour 1 : 0, 2
        Tour 2 : 6, 2
        Tour 3 : 10, 2

    section P2 (durée 4)
        Tour 1 : 2, 2
        Tour 2 : 8, 2

    section P3 (durée 2)
        Tour 1 : 4, 2

    section P4 (durée 2)
        Tour 1 : 12, 2
```

Principe : Chaque processus reçoit un **quantum de temps** fixe (ex: 10 ms). Si le processus ne se termine pas dans son quantum, il est remis dans la file des prêts et le suivant est exécuté.

Avantages : Équitable, temps de réponse court pour les processus interactifs.

Inconvénient : Le choix du quantum est crucial : trop court ⇒ changement de contexte trop fréquent, trop long ⇒ dégradation du temps de réponse.

Exemple de calcul (Round Robin, quantum = 2) :

Processus	Durée	Début	Fin	Temps d'attente
P1	6	0	12	6
P2	4	2	10	4

P3	2	4	6	2
P4	2	12	14	10

Temps d'attente moyen : $(6 + 4 + 2 + 10) / 4 = 5,5$ unitÃ©s de temps.

4. Syst me de fichiers

4.1 Arborescence des fichiers

```
graph TB
    ROOT["/ (racine)"] --> BIN["/bin<br/>commandes essentielles"]
    ROOT --> HOME["/home<br/>rÃ©pertoires utilisateurs"]
    ROOT --> ETC["/etc<br/>configuration systÃ¨me"]
    ROOT --> USR["/usr<br/>programmes installÃ©s"]
    ROOT --> VAR["/var<br/>donnÃ©es variables (logs)"]
    ROOT --> TMP["/tmp<br/>fichiers temporaires"]
    ROOT --> DEV["/dev<br/>pÃ©riphÃ©riques"]

    HOME --> USER1["/home/madani<br/>(votre rÃ©pertoire)"]
    USER1 --> DOC["Documents"]
    USER1 --> TEL["TÃ©lÃ©chargements"]
    USER1 --> BUR["Bureau"]
    USER1 --> MUSIC["Musique"]
    USER1 --> IMG["Images"]

    style ROOT fill:#e74c3c,color:#fff
    style HOME fill:#3498db,color:#fff
    style USER1 fill:#2ecc71,color:#fff
```

Structure : Arborescence enracinée (Unix/Linux) ou avec lettres de lecteurs (Windows).

Chemin absolu : commence par / (racine). Ex: /home/madani/Documents/cours.md

Chemin relatif : par rapport au répertoire courant. Ex: ../Documents/cours.md

Raccourcis :

- `.` : répertoire courant
- `..` : répertoire parent
- `~` : répertoire personnel de l'utilisateur

4.2 Commandes de base (terminal Linux)

Commande	R��le	Exemple
pwd	Afficher le r��pertoire courant	pwd ��t' <font face='DocMono'
ls	Lister les fichiers	ls -la (d��taill��, cach��s)
cd	Changer de r��pertoire	cd Documents/cours
mkdir	Cr��er un r��pertoire	mkdir nouveau_dossier
cp	Copier un fichier	cp source.txt dest.txt
mv	D��placer/renommer	mv ancien.txt nouveau.txt
rm	Supprimer	rm fichier.txt (attention !)
cat	Afficher le contenu	cat fichier.txt
chmod	Modifier les permissions	chmod 755 script.sh

4.3 Permissions (Unix)

```
graph TD
    subgraph "Permissions d'un fichier"
        PERM["-rwxr-xr--"]
        TYP["Type :<br/>- fichier<br/>d rÃ©pertoire<br/>l lien"]
        U["User (owner)<br/>rwx<br/>(lecture, Ã©criture, exÃ©cution)"]
        G["Group<br/>r-x<br/>(lecture, exÃ©cution)"]
        O["Others<br/>r--<br/>(lecture seule)"]
    end

    TYP --> PERM
    PERM --> U --> G --> O
```

ReprÃ©sentation symbolique : -rwxr-xr--

- Position 1 : type (– fichier, d rÃ©pertoire, l lien symbolique)
- Positions 2-4 : droits du **propriÃ©taire** (r=read, w=write, x=execute)
- Positions 5-7 : droits du **groupe**
- Positions 8-10 : droits des **autres**

ReprÃ©sentation octale :

- r = 4, w = 2, x = 1
- `chmod 755 â†’ rwxr-xr-x`
- `chmod 644 â†’ rw-r--r--`

Exemple pour l'enseignement : ProtÃ©ger un fichier de notes confidentiel :

```
chmod 600 notes_eleves.txt # seul le propriÃ©taire peut lire/Ã©crire
```

QCM d'auto-Ã©valuation

1. Quel est le rÃ´le principal du noyau (kernel) d'un OS ?

- a) Afficher une interface graphique b) GÃ©rer les ressources matÃ©rielles â€œ c) Naviguer sur Internet

2. Dans quel Ã©tat un processus attend-il que le CPU lui soit allouÃ© ?

- a) Running b) Waiting c) Ready â€œ

3. Quel algorithme d'ordonnancement utilise un quantum de temps fixe ?

- a) FIFO b) Round Robin â€œ c) SJF

4. Que signifie le droit x sur un rÃ©pertoire ?

- a) Pouvoir le supprimer b) Pouvoir le traverser (y accÃ©der) â€œ c) Pouvoir le renommer

5. Quelle commande permet d'afficher le rÃ©pertoire courant sous Linux ?

- a) cd b) ls c) pwd â€œ

Conclusion

Le systÃ¨me d'exploitation est un logiciel fondamental qui gÃ©re toutes les ressources de l'ordinateur. La distinction noyau/espace utilisateur garantit la stabilitÃ© et la sÃ©curitÃ©. Les processus et threads sont les unitÃ©s d'exÃ©cution de base ; leur ordonnancement (FIFO, Round Robin) dÃ©termine la

l'activité du système. Le système de fichiers organise les données dans une arborescence et les permissions Unix permettent de contrôler les accès.

Prochaine séance : Algorithmique fondamentale – variables, conditions, boucles et complexité.

Note pédagogique ENS

Transposition didactique : En classe de SNT Seconde (thème « Systèmes d'exploitation ») :

- Simuler l'ordonnancement Round Robin en faisant jouer les processus par des élèves. Un élève est l'ordonnanceur et distribue un « quantum » (ex: 30 secondes) à chaque processus.
- Activité : créer une arborescence de répertoires pour un projet de classe (avec `mkdir` en ligne de commande)
- Expliquer les permissions avec l'analogie d'une salle de classe (propriétaire = enseignant, groupe = élèves)

Lien avec les langues : Le concept d'arborescence de fichiers peut être comparé à la structure grammaticale (arbres syntaxiques). Les threads (exécution logée) peuvent être comparés aux conversations parallèles dans une classe.

Conseil pour l'enseignant formateur : Insister sur la différence entre processus et programme (concept souvent confondu par les débutants). Montrer le gestionnaire de tâches (Windows) ou le moniteur système (Linux) en direct pendant le cours pour illustrer les processus actifs. Proposer aux étudiants de créer un petit script Python qui liste les processus système avec la bibliothèque `psutil`.

Prolongement : Les étudiants peuvent explorer les conteneurs Docker comme évolution des machines virtuelles, ou étudier le noyau Linux et la gestion des processus en ligne de commande avec les outils `ps`, `top`, `htop`, `kill`. Proposer une réflexion sur l'impact des systèmes d'exploitation dans l'éducation (choix entre Windows, macOS, Linux, ChromeOS dans les établissements scolaires).

Bibliographie

- Tanenbaum, A. & Bos, H. (2015). *Systèmes d'exploitation : Modern Operating Systems* (4e éd.). Pearson.
- Silberschatz, A., Galvin, P. & Gagne, G. (2018). *Operating System Concepts* (10e éd.). Wiley.
- Le livre du système Linux : <https://linuxcommand.org/>
- Dowek, G. et al. (2019). *SNT Seconde*. Hachette (chapitre « Systèmes d'exploitation »).
- Documentation Ubuntu : <https://doc.ubuntu-fr.org/>

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_05.pdf

Cours Informatique à l'ENS 1^{ère} année S^éance 05 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'1^{ère} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Définir un algorithme et énumérer ses 5 caractéristiques fondamentales
- Utiliser des variables, des types de données et des expressions dans un pseudo-code
- Créer des structures conditionnelles (si/sinon/selon)
- Utiliser des boucles conditionnelles (tant que) et comptées (pour)
- Analyser la complexité temporelle d'un algorithme simple ($O(1)$, $O(n)$, $O(n^2)$)
- Tracer manuellement l'exécution d'un algorithme pas à pas

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Qu'est-ce qu'un algorithme ? Définition et caractéristiques (10 min)
2. Variables, types et expressions (15 min)
3. Structures conditionnelles : si, sinon, selon (15 min)
4. Boucles : tant que, pour (20 min)
5. Complexité algorithmique : $O(1)$, $O(n)$, $O(n^2)$ (15 min)
6. Trace d'exécution manuelle (10 min)

7. Applications pédagogiques pour SNT/NSI (5 min)

1. Qu'est-ce qu'un algorithme ?

Définition : Un algorithme est une suite **finie** d'instructions **non ambiguës** qui, étant données des **entrées**, produit des **sorties** et se **termine** en un temps fini.

5 caractéristiques fondamentales

1. **Finitude** : l'algorithme se termine après un nombre fini d'étapes (pas de boucle infinie)
2. **Non-ambiguïté** : chaque instruction est claire et sans interprétation multiple
3. **Entrées** : zéro ou plusieurs données d'entrée
4. **Sorties** : un ou plusieurs résultats produits
5. **Efficacité** : l'algorithme utilise des ressources (temps, mémoire) raisonnables

Exemple fondateur : algorithme d'Euclide (PGCD)

Origine : Du mathématicien perse Al-Khwārizmī (IXe s.).

```

Algorithme PGCD(a, b)
TANT QUE b ≠ 0 FAIRE
  r ← a MOD b; a ← b; b ← r
FIN TANT QUE
RETOURNER a

graph TD
DEBUT("Début") --> ENTREE["Entrer a, b"]
ENTREE --> COND{"b = 0 ?"}
COND -- OUI --> SORTIE["Afficher a (PGCD)"]
SORTIE --> FIN("Fin")
COND -- NON --> CALC["r ← a MOD b<br/>a ← b<br/>b ← r"]
CALC --> COND

style DEBUT fill:#2ecc71,color:#fff
style ENTREE fill:#3498db,color:#fff
style COND fill:#f39c12,color:#000
style SORTIE fill:#3498db,color:#fff
style FIN fill:#e74c3c,color:#fff
style CALC fill:#9b59b6,color:#fff

```

2. Variables, types et expressions

Une **variable** est un espace mémoire nommé contenant une valeur qui peut changer.

```

age ← 25
prénom ← "Alice"
est_majeur ← VRAI

```

Type	Signification	Exemples
Entier (int)	Nombre entier	42, -3, 0
Réel (float)	Nombre à virgule	3.14, -0.5
Booléen (bool)	Vrai ou Faux	VRAI, FAUX
Caractère (char)	Un seul caractère	'A', 'À'
Chaîne (string)	Texte	"Bonjour"

Opérateurs :

- **Arithmétiques** : +, -, *, /, MOD (reste), DIV (division entière)
- **Comparaison** : =, <, >, <=, >=
- **Logiques** : ET (AND), OU (OR), NON (NOT)

```
moyenne ← (note1 + note2 + note3) / 3
est_pair ← (x MOD 2 = 0)
majeur ← (Age ≥ 18) ET (nationalité = "française")
```

3. Structures conditionnelles

3.1 Si / Sinon (if / else)

```
graph TD
    COND2{"Condition ?"} -- VRAI --> B1["Bloc d'instructions 1"]
    COND2 -- FAUX --> B2["Bloc d'instructions 2"]
    B1 --> SUITE
    B2 --> SUITE
    style COND2 fill:#f39c12,color:#000

    SI note ≥ 10 ALORS
    AFFICHER "Admis"
    SINON
    AFFICHER "Ajourné"
    FIN SI
```

Avec SI/SINON SI :

```
SI note ≥ 16 : "Très bien" ; SINON SI note ≥ 14 : "Bien"
SINON SI note ≥ 12 : "Assez bien" ; SINON SI note ≥ 10 : "Passable" ; SINON : "Ajourné"
FIN SI
```

3.2 Selon / Cas (switch / case)

```
SELON note FAIRE
CAS "A": "Excellent" ; CAS "B": "Bien" ; CAS "C": "Assez bien"
CAS "D": "Passable" ; CAS "E": "Insuffisant" ; DEFAULT: "Invalide"
FIN SELON
```

4. Boucles

4.1 Boucle TANT QUE (while)

Réalise un bloc d'instructions **tant que** la condition est vraie. Le test est effectué avant chaque itération.

```
i ← 1
TANT QUE i ≤ 10 FAIRE
AFFICHER i
i ← i + 1
FIN TANT QUE
```

Exemple de somme des N premiers entiers :

```
Algorithme Somme(N)
somme ← 0; i ← 1
TANT QUE i ≤ N FAIRE
somme ← somme + i; i ← i + 1
FIN TANT QUE
```

RETOURNER somme

4.2 Boucle POUR (for)

Réalise un bloc un nombre d de fois. Le compteur est géré automatiquement.

```
POUR i DE 1 À N FAIRE
  AFFICHER "Itération ", i
FIN POUR
```

Exemple de table de multiplication :

```
Algorithme TableMultiplication(N)
POUR i DE 1 À 10 FAIRE
  AFFICHER N, " - ", i, " = ", N - i
FIN POUR

graph TD
  DEBUT2("Début") --> INIT["i ← 1"]
  INIT --> COND3{"i ≤ N ?"}
  COND3 -- OUI --> CORPS["Afficher i<br/>i + 1"]
  CORPS --> COND3
  COND3 -- NON --> FIN2("Fin")
  style DEBUT2 fill:#2ecc71,color:#fff
  style COND3 fill:#f39c12,color:#000
  style CORPS fill:#3498db,color:#fff
  style FIN2 fill:#e74c3c,color:#fff
```

Situation	Boucle recommandée
Nombre d'itérations connu à l'avance	POUR
Parcours d'un tableau de taille N	POUR
Attente d'une condition externe	TANT QUE
Saisie utilisateur jusqu'à valeur sentinelle	TANT QUE

5. Complexité algorithmique

La **complexité** mesure le nombre d'opérations élémentaires en fonction de la taille n de l'entrée. On s'intéresse au comportement asymptotique (grand n).

```
graph LR
  subgraph "Complexités classiques"
    O1["O(1) ← Constant<br/>Accès direct"]
    O2["O(log n) ← Logarithmique<br/>Dichotomie"]
    O3["O(n) ← Linéaire<br/>Parcours simple"]
    O4["O(n log n) ← Quasi-linéaire<br/>Tri fusion"]
    O5["O(n²) ← Quadratique<br/>Boucles imbriquées"]
    O6["O(2ⁿ) ← Exponentiel<br/>Explosion combinatoire"]
  end
  O1 --> O2 --> O3 --> O4 --> O5 --> O6
  style O1 fill:#2ecc71,color:#fff
  style O3 fill:#3498db,color:#fff
  style O5 fill:#e67e22,color:#fff
  style O6 fill:#e74c3c,color:#fff
```

O(1) ← Constant : RETOURNER T[1] (1 opération, quel que soit n).

O(n) ← Linéaire :

```

Algorithme SommeTableau(T, n)
somme ← 0
POUR i DE 1 À n FAIRE
somme ← somme + T[i]
FIN POUR
RETOURNER somme // n additions → O(n)

```

$O(n^2)$ Quadratique :

```

Algorithme TriBulles(T, n)
POUR i DE 1 À n-1 FAIRE
POUR j DE 1 À n-i FAIRE
SI T[j] > T[j+1] ALORS
Échanger T[j] et T[j+1]
FIN SI
FIN POUR
FIN POUR // n(n-1)/2 comparaisons → O(n^2)

```

```

graph TD
subgraph "Croissance comparée"
L1["n=10 : O(1)=1, O(n)=10, O(n^2)=100"]
L2["n=20 : O(1)=1, O(n)=20, O(n^2)=400"]
L3["n=50 : O(1)=1, O(n)=50, O(n^2)=2500"]
L4["n=100 : O(1)=1, O(n)=100, O(n^2)=10000"]
end
style L1 fill:#2ecc71,color:#fff
style L2 fill:#3498db,color:#fff
style L3 fill:#e67e22,color:#fff
style L4 fill:#e74c3c,color:#fff

```

Règle : $O(n)$ est utilisable pour de grandes données, $O(n^2)$ est limite, $O(2^n)$ est inexploitable.

6. Trace d'exécution manuelle

La trace permet de suivre les variables pas à pas pour comprendre et vérifier un algorithme.

Algorithme Factorielle de N :

```

Algorithme Factorielle(N)
resultat ← 1; i ← 1
TANT QUE i ≤ N FAIRE
resultat ← resultat * i
i ← i + 1
FIN TANT QUE
RETOURNER resultat

```

Trace pour N = 4 : (i, resultat) : (1, 1) → (2, 2) → (3, 6) → (4, 24) → (5, FAUX).

Résultat : Factorielle(4) = 24

QCM d'auto-évaluation

- Quelle est la complexité d'une recherche séquentielle dans un tableau de taille n ?
a) $O(1)$ b) $O(n)$ c) $O(n^2)$
- Quelle boucle utiliser quand le nombre d'itérations est connu à l'avance ?
a) TANT QUE b) POUR c) RÉPÉTER
- Quel est le résultat de l'algorithme d'Euclide pour PGCD(12, 8) ?
a) 4 b) 2 c) 6

4. Quelle caractéristique garantit qu'un algorithme ne boucle pas indéfiniment ?

- a) Non-ambiguïté b) Finitude c) Efficacité

5. Combien d'opérations un tri bulles effectue-t-il pour $n = 10$?

- a) 10 b) Environ 45 c) 100

Conclusion

L'algorithmique est la science de la résolution de problèmes par des procédures pas à pas. Variables, conditions et boucles sont les briques fondamentales. La complexité ($O(1)$, $O(n)$, $O(n^2)$) permet de comparer les algorithmes. La trace d'exécution est un outil pédagogique essentiel pour déboguer.

Prochaine séance : Structures de données : tableaux, piles LIFO, files FIFO, listes chaînées.

Note pédagogique ENS

Transposition didactique : En SNT Seconde et NSI Première :

- **Activité débranchée** : Les élèves suivent un algorithme sur papier quadrillé servant de mémoire
- **Recette de cuisine** : Ingrédients = variables, Étapes = instructions, Répétitions = boucles
- **Tri de cartes** : Faire trier des cartes numérotées en suivant le tri bulles ou tri par sélection
- **Complexité** : Compter le nombre de comparaisons pour différentes tailles de jeu

Lien avec les langues : L'algorithmique peut être comparée à l'analyse grammaticale (une phrase se décompose en sujets/verbes/compléments comme des instructions) et à la rédaction d'un plan de dissertation (structure séquentielle avec des conditions).

Conseil pour l'enseignant formateur : L'algorithmique est le socle de toute la pensée informatique. Insister sur la phase de conception (pseudo-code, organigramme) avant le codage. Utiliser des quiz interactifs (Kahoot!, Plickers) pour vérifier la compréhension des concepts (variables, conditions, boucles) de façon ludique. Proposer des défis de programmation sur des plateformes comme France-ioi ou CodinGame.

Prolongement : Les étudiants peuvent approfondir avec des algorithmes plus avancés (récursivité, diviser-pour-régner, glouton) et leur application en NSI Terminale. Proposer une activité complémentaire : implémenter le tri fusion en Python et comparer expérimentalement sa complexité avec le tri bulles (visualisation avec matplotlib).

Bibliographie

- Cormen, T. et al. (2009). *Introduction to Algorithms* (3e éd.). MIT Press.
- Dowek, G. et al. (2019). *SNT Seconde et NSI Première*. Hachette.
- Delannoy, C. (2020). *Algorithmique et programmation en Python*. Eyrolles.
- Biernat, J. et al. (2020). *Algorithmique : cours et exercices corrigés*. Dunod.
- Al-Khwarizmi (IXe s.). *Al-Jabr wa al-Muqabala*.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Templatte/Presentation_BELACEL_V3.pptx

PDF: Cours_Informatique_ENS_1ère_année_FR_06.pdf

Cours Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 06 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques spÃ©cifiques

Ã€ l'issue de cette sÃ©ance, l'Ã©tudiant est capable de :

- DÃ©finir une structure de donnÃ©es et expliquer son rÃ´le algorithmique
- Distinguer accÃ¨s direct ($O(1)$) et accÃ¨s sÃ©quentiel ($O(n)$) dans un tableau
- ImplÃ©menter les opÃ©rations push, pop, peek d'une pile (LIFO) et enqueue, dequeue d'une file (FIFO)
- ModÃ©liser une liste chaÃªnÃ©e avec nÃuds et pointeurs
- Choisir la structure adaptÃ©e (tableau, pile, file, liste chaÃªnÃ©e) selon le problÃªme

Contexte rÃ©glementaire algÃ©rien

Ce cours s'inscrit dans le cadre rÃ©glementaire algÃ©rien de formation des enseignants :

- **ArrÃªtÃ© ministÃ©riel portant crÃ©ation des ENS (cycle Licence/Master)** â€” dÃ©finit le cadre institutionnel des Ã©coles Normales SupÃ©rieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algÃ©riennes** â€” fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **ArrÃªtÃ© 712 fixant le programme des concours ENS** â€” dÃ©termine les Ã©preuves et les programmes des concours d'accÃ¨s aux ENS, en cohÃ©rence avec les contenus enseignÃ©s.
- **RÃ©fÃ©rentiel des compÃ©tences numÃ©riques algÃ©rien** â€” cadre national (en cours d'Ã©laboration) dÃ©finissant les compÃ©tences numÃ©riques attendues des enseignants.
- **Benchmark des 10 universitÃ©s algÃ©riennes** â€” Ã©tude comparative des pratiques d'enseignement de l'informatique, consultÃ©e pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Notion de structure de donnÃ©es â€” 5 min
2. Tableaux (array) â€” 15 min
3. Piles LIFO â€” 20 min
4. Files FIFO â€” 20 min
5. Listes chaÃªnÃ©es â€” 15 min
6. Comparaison tableau vs liste chaÃªnÃ©e â€” 10 min

7. QCM et conclusion 5 min

1. Notion de structure de données (5 min)

Une **structure de données** est une organisation logique des données en mémoire permettant des opérations efficaces (ajout, suppression, recherche, parcours).

Critères de choix : type d'accès (direct/séquentiel), dynamisme (taille fixe/variable), type d'opérations fréquentes.

2. Tableaux (array) (15 min)

Un **tableau** est une zone mémoire contiguë contenant des éléments de même type, chacun repéré par un indice.

```
// Pseudo-code : déclaration et accès
notes: TABLEAU[10] D'ENTIERS
notes[0] ← 15
notes[1] ← 12
notes[2] ← 18

// Parcours et somme
somme ← 0
POUR i DE 0 À 9 FAIRE
  somme ← somme + notes[i]
FIN POUR
```

Complexités fondamentales :

- **Accès par indice : $O(1)$** 5 min calcul direct de l'adresse mémoire : $addr = base + i \times \text{taille_élément}$
- **Parcours séquentiel : $O(n)$** 5 min il faut visiter chaque élément
- **Insertion au milieu : $O(n)$** 5 min décalage des éléments suivants

Propriétés : taille fixe (statique), contiguë en mémoire, accès aléatoire instantané.

Exemple pédagogique NSI : un tableau de 30 notes pour une classe. Calcul de la moyenne en $O(n)$. Recherche du maximum en $O(n)$.

```
graph LR
  subgraph "Tableau notes[5] en mémoire"
    A["Indice 0<br/>15"]
    B["Indice 1<br/>12"]
    C["Indice 2<br/>18"]
    D["Indice 3<br/>10"]
    E["Indice 4<br/>14"]
  end
  ACCES["Accès notes[2] ← O(1)<br/>Adresse = base + 2×4 octets"] --> C

  style A fill:#3498db,color:#fff
  style B fill:#3498db,color:#fff
  style C fill:#e74c3c,color:#fff
  style D fill:#3498db,color:#fff
  style E fill:#3498db,color:#fff
  style ACCES fill:#2ecc71,color:#fff
```

3. Piles 5 min LIFO (20 min)

Principe : *Last In, First Out* "le dernier Ã©lÃ©ment ajoutÃ© est le premier retirÃ©.

Analogie : une pile d'assiettes. On pose la nouvelle assiette sur le dessus ; on prend toujours celle du dessus.

```
// OpÃ©rations sur une pile
PILE pile â†• VIDE

empiler(pile, 10) // [10]
empiler(pile, 20) // [10, 20]
empiler(pile, 30) // [10, 20, 30]
x â†• depiler(pile) // x = 30, pile = [10, 20]
y â†• depiler(pile) // y = 20, pile = [10]
```

OpÃ©ration	RÃle	ComplexitÃ©
<code>push(v)</code>	Ajouter au sommet	$O(1)$
<code>pop()</code>	Retirer et renvoyer le sommet	$O(1)$
<code>peek()</code>	Consulter le sommet sans retirer	$O(1)$
<code>is_empty()</code>	VÃ©rifier si la pile est vide	$O(1)$

```
graph TB
    subgraph "Pile LIFO â€” Ã©tat aprÃ¨s push(10), push(20), push(30)"
        S3["30 â†• SOMMET (push / pop)"]
        S2["20"]
        S1["10 â†• BASE"]
    end

    subgraph "OpÃ©rations"
        P1["push(40) â†• [10, 20, 30, 40]"]
        P2["pop() â†• retourne 30 â†• [10, 20]"]
        P3["peek() â†• retourne 30 (sans retirer)"]
    end

    S3 --> P1
    S3 --> P2
    S3 --> P3

    style S3 fill:#e74c3c,color:#fff
    style S2 fill:#f39c12,color:#fff
    style S1 fill:#3498db,color:#fff
    style P1 fill:#2ecc71,color:#fff
    style P2 fill:#2ecc71,color:#fff
    style P3 fill:#2ecc71,color:#fff
```

Exemples concrets :

- **Ctrl+Z** (annulation) : chaque action est empilÃ©e, Ctrl+Z dÃ©pile
- **Historique navigateur** : bouton "PrÃ©cÃ©dent" dÃ©pile les URLs visitÃ©es
- **Pile d'appels (call stack)** : le systÃ©me empile les appels de fonctions

Enseignement NSI : la pile d'appels explique la rÃ©cursivitÃ©. Chaque appel rÃ©cursif empile un nouveau cadre ; le retour dÃ©pile.

4. Files "FIFO" (20 min)

Principe : *First In, First Out* "le premier Ã©lÃ©ment ajoutÃ© est le premier retirÃ©.

Analogie : une file d'attente Ã la cantine. Le premier arrivÃ© est le premier servi.

```
// Opérations sur une file
FILE file ← VIDE

enfiler(file, 10) // [10]
enfiler(file, 20) // [10, 20]
enfiler(file, 30) // [10, 20, 30]
x ← defiler(file) // x = 10, file = [20, 30]
y ← defiler(file) // y = 20, file = [30]
```

Opération	R��le	Complexit��
<code>enqueue(v)</code>	Ajouter �� la fin (queue)	$O(1)$
<code>dequeue()</code>	Retirer et renvoyer le d��but (t��te)	$O(1)$
<code>front()</code>	Consulter la t��te sans retirer	$O(1)$
<code>is_empty()</code>	V��rifier si la file est vide	$O(1)$

```
graph LR
    subgraph "File FIFO"
        direction LR
        TETE["T  TE<br/>(dequeue)"]
        F1["10"]
        F2["20"]
        F3["30"]
        QUEUE["QUEUE<br/>(enqueue)"]
    end

    TETE --> F1
    F1 --> F2
    F2 --> F3
    F3 --> QUEUE

    ENTREE["enqueue(40)   ' [10, 20, 30, 40]"] -.-> QUEUE
    TETE -.-> SORTIE["dequeue()   ' 10   ' [20, 30]"]

    style F1 fill:#e74c3c,color:#fff
    style F2 fill:#f39c12,color:#fff
    style F3 fill:#3498db,color:#fff
    style TETE fill:#2ecc71,color:#fff
    style QUEUE fill:#2ecc71,color:#fff
    style ENTREE fill:#9b59b6,color:#fff
    style SORTIE fill:#9b59b6,color:#fff
```

Exemples concrets :

- **File d'impression** : les documents s'impriment dans l'ordre d'envoi
- **Tampon clavier** : les touches sont trait  es dans l'ordre de frappe
- **Gestionnaire de t  ches** : files d'attente des processus CPU

Enseignement SNT : simuler une file d'attente avec les   l  ves (jeu de r  le). Chaque   l  ve re  oit un ticket num  rot  .

5. Listes cha  n  es (15 min)

Une **liste cha  n  e** est une structure dynamique o   chaque   l  ment (n  ud) contient une valeur et un pointeur vers le n  ud suivant.

```
// Structure d'un n  ud
Noeud {
    valeur: ENTIER
```

```

suivant: POINTEUR_VERS_Noed
}

graph LR
subgraph "Liste chaînée simple"
N1["Nœud 1<br/>valeur: 10<br/>suivant: *"]
N2["Nœud 2<br/>valeur: 20<br/>suivant: *"]
N3["Nœud 3<br/>valeur: 30<br/>suivant: null"]
end

TETE["Tête"] --> N1
N1 -->|"pointeur"| N2
N2 -->|"pointeur"| N3
N3 --> FIN["null (fin)"]

style N1 fill:#3498db,color:#fff
style N2 fill:#2ecc71,color:#fff
style N3 fill:#f39c12,color:#fff
style TETE fill:#e74c3c,color:#fff
style FIN fill:#95a5a6,color:#fff

```

Caractéristiques :

- **Taille dynamique** : pas de limite fixe, la mémoire est allouée au fur et à mesure
- **Insertion au début** : $O(1)$ car il suffit de modifier le pointeur de tête
- **Insertion au milieu** : $O(1)$ car si on a le pointeur vers le nœud précédent
- **Accès par indice** : $O(n)$ car obligé de parcourir depuis la tête
- **Recherche** : $O(n)$ car parcours séquentiel obligatoire

Pseudo-code insertion au début :

```

// Insérer valeur au début d'une liste chaînée
FONCTION inserer_debut(tete, valeur)
nouveau ← NOUVEAU_Noed(valeur)
nouveau.suivant ← tete
RETOURNER nouveau // nouvelle tête
FIN FONCTION

```

6. Comparaison et choix (10 min)

```

graph TB
subgraph "Comparaison Tableau vs Liste chaînée"
subgraph "Tableau"
T1["Accès  $O(1)$  Mémoire contiguë<br/> Insertion au début  $O(n)$ <br/> Taille fixe"]
end
subgraph "Liste chaînée"
L1["Insertion  $O(1)$ <br/> Taille dynamique<br/> Accès  $O(n)$ <br/> Mémoire dispersée"]
end
end

style T1 fill:#3498db,color:#fff
style L1 fill:#2ecc71,color:#fff

```

Critère	Tableau	Liste chaînée
Accès par indice	$O(1)$ direct	$O(n)$ séquentiel

Insertion d'Ã©lÃ©ment	$O(n)$ Ã© d'Ã©calage	$O(1)$ Ã© pointeur
Insertion fin	$O(1)$ Ã© si place	$O(1)$ Ã© si queue connue
Suppression	$O(n)$ Ã© d'Ã©calage	$O(1)$ Ã© pointeur
Recherche	$O(n)$	$O(n)$
MÃ©moire	Fixe, contiguÃ©	Dynamique, dispersÃ©e
Cache CPU	Favorable (localitÃ©)	DÃ©favorable

Guide de choix :

- **AccÃ©s frÃ©quent par position** Ã© Tableau (ex: liste de notes indicÃ©es)
- **Insertions/suppressions frÃ©quentes au dÃ©but ou au milieu** Ã© Liste chaÃ©nÃ©e
- **OpÃ©rations LIFO** Ã© Pile (ex: historique, annulation)
- **OpÃ©rations FIFO** Ã© File (ex: file d'attente, tampon)
- **Taille inconnue Ã© l'avance** Ã© Liste chaÃ©nÃ©e

QCM d'auto-Ã©valuation

1. Quelle est la complexitÃ© d'accÃ©s Ã© un Ã©lÃ©ment dans un tableau par son indice ?

a) $O(n)$ b) $O(1)$ Ã© c) $O(n^2)$

2. Dans une pile LIFO, quel Ã©lÃ©ment retire-t-on avec `pop()` ?

a) Le premier ajoutÃ© b) Le dernier ajoutÃ© Ã© c) L'Ã©lÃ©ment au milieu

3. Quelle analogie reprÃ©sente le mieux une file FIFO ?

a) Une pile d'assiettes b) Une file d'attente au guichet Ã© c) Un tableau d'affichage

4. Dans une liste chaÃ©nÃ©e, quel est le coÃ»t d'une insertion au dÃ©but ?

a) $O(1)$ Ã© b) $O(n)$ c) $O(n^2)$

5. Quelle structure de donnÃ©es utilise le bouton Ã© PrÃ©cÃ©dent Ã© du navigateur ?

a) File b) Pile Ã© c) Liste chaÃ©nÃ©e

Conclusion (5 min)

Les structures de donnÃ©es sont des outils algorithmiques essentiels. Un tableau favorise l'accÃ©s rapide ($O(1)$). Une liste chaÃ©nÃ©e favorise l'insertion dynamique ($O(1)$). La pile organise le Ã© dernier arrivÃ©, premier servi Ã© (LIFO) ; la file organise le Ã© premier arrivÃ©, premier servi Ã© (FIFO). Le choix de la structure dÃ©termine l'efficacitÃ© du programme.

Prochain cours : RÃ©seaux et Internet Ã© comprendre comment les ordinateurs communiquent.

Note pÃ©dagogique ENS

Transposition didactique :

- **SNT Seconde** : utiliser des activitÃ©s dÃ©branchÃ©es pour piles (assiettes, livres) et files (file d'attente physique)

- **NSI Premi re** : impl menter une pile en Python avec une liste ; visualiser la pile d'appels r cursifs
- **Lien interdisciplinaire** : en math matiques, la pile sert    valuer les expressions arithm tiques (notation polonaise inverse)

Conseil pour l'enseignant formateur : Faire tracer l' volution de la pile et de la file au tableau  tape par  tape. Chaque  l ve manipule des post-its repr sentant les n uds.

Prolongement : Les  tudiants peuvent approfondir en d couvrant les structures de donn es avanc es : arbres binaires de recherche, graphes, tables de hachage. Proposer une activit  compl mentaire en NSI : impl menter un correcteur orthographique simple avec une table de hachage pour v rifier les mots d'un texte fran ais.

Bibliographie

- Cormen, Leiserson, Rivest & Stein    Introduction to Algorithms (ch. 10 : Elementary Data Structures)
- Aho, Ullman    Data Structures and Algorithms
- Downey    Think Python (ch. 17 : Stacks and Queues)
- Documentation Python : module `queue` (impl mentations thread-safe)

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/Pg-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1 re_ann e_FR_07.pdf

Cours Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 07 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques spÃ©cifiques

Ã€ l'issue de cette sÃ©ance, l'Ã©tudiant est capable de :

- Distinguer LAN, WAN, WLAN et donner des exemples concrets
- Lire et interprÃ©ter une adresse IPv4 et son masque de sous-rÃ©seau
- Expliquer le rÃ´le du DNS dans la rÃ©solution de noms
- DÃ©crire le modÃ¨le client-serveur avec un exemple du Web
- DiffÃ©rencier les protocoles HTTP, HTTPS, FTP, SMTP, IMAP, POP3
- Comprendre le principe du cloud computing et ses usages pÃ©dagogiques

Contexte rÃ©glementaire algÃ©rien

Ce cours s'inscrit dans le cadre rÃ©glementaire algÃ©rien de formation des enseignants :

- **ArrÃªtÃ© ministÃ©riel portant crÃ©ation des ENS (cycle Licence/Master)** â€” dÃ©finit le cadre institutionnel des Ã©coles Normales SupÃ©rieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algÃ©riennes** â€” fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **ArrÃªtÃ© 712 fixant le programme des concours ENS** â€” dÃ©termine les Ã©preuves et les programmes des concours d'accÃ©s aux ENS, en cohÃ©rence avec les contenus enseignÃ©s.
- **RÃ©fÃ©rentiel des compÃ©tences numÃ©riques algÃ©rien** â€” cadre national (en cours d'Ã©laboration) dÃ©finissant les compÃ©tences numÃ©riques attendues des enseignants.
- **Benchmark des 10 universitÃ©s algÃ©riennes** â€” Ã©tude comparative des pratiques d'enseignement de l'informatique, consultÃ©e pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Types de rÃ©seaux (LAN, WAN, WLAN) â€” 10 min
2. Adresses IP, masque et sous-rÃ©seau â€” 15 min
3. DNS : l'annuaire d'Internet â€” 15 min
4. ModÃ¨le client-serveur â€” 15 min
5. Protocoles du Web et de la messagerie â€” 15 min
6. Cloud computing â€” 10 min

7. QCM et conclusion â€” 5 min

1. Types de réseaux (10 min)

Un **réseau informatique** est un ensemble d'appareils interconnectés qui échangent des données.

Type	Portée	Exemple
LAN (Local Area Network)	Bâtiment, salle	Réseau de la salle informatique
WAN (Wide Area Network)	Région, pays, monde	Internet
WLAN (Wireless LAN)	Local, sans fil	Wi-Fi de l'établissement
MAN (Metropolitan Area Network)	Ville	Réseau universitaire urbain
PAN (Personal Area Network)	~10 m	Bluetooth, smartphone

```

graph TD
    subgraph "Topologie d'un réseau typique"
        INTERNET[INTERNET<br/>WAN mondial]
        FAI[FAI<br/>Fournisseur d'accès]
        ROUT[Routeur<br/>ADSL / Fibre]

        ROUT --> SW1[Switch<br/>Réseau filaire]
        ROUT --> AP[Point d'accès<br/>Wi-Fi - WLAN]

        SW1 --> PC1[PC fixe 1]
        SW1 --> PC2[PC fixe 2]
        SW1 --> SRV[Serveur<br/>pédagogique]

        AP --> TEL[Smartphone]
        AP --> TAB[Tablette]
        AP --> PC3[PC portable]
    end

    INTERNET --> FAI
    FAI --> ROUT

    style INTERNET fill:#e74c3c,color:#fff
    style FAI fill:#9b59b6,color:#fff
    style ROUT fill:#3498db,color:#fff
    style SW1 fill:#2ecc71,color:#fff
    style AP fill:#f39c12,color:#fff
    style SRV fill:#1abc9c,color:#fff

```

Éléments matériels :

- **Routeur** : interconnecte les réseaux, achemine les paquets
- **Switch** : connecte les machines d'un même réseau local
- **Point d'accès** : extension Wi-Fi du réseau filaire
- **Câble Ethernet (RJ45)** : connexion filaire jusqu'à 1 Gbit/s

Exemple pédagogique : dans une salle de classe, le réseau local (LAN) relie les PC via un switch. Le routeur donne accès à Internet (WAN) via le FAI. Le Wi-Fi (WLAN) permet aux tablettes de se connecter sans fil.

2. Adresses IP et masque (15 min)

Une **adresse IP** est un identifiant unique attribu   chaque appareil connect   un r  seau.

IPv4 (32 bits)

Format : 4 nombres d'octets de 0 à 255, séparés par des points.

192.168.1.10 " adresse priv e typique d'un r seau local

Classe d'adresses priv es : 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

IPv6 (128 bits)

Format : 8 groupes hexadécimaux de 16 bits, séparés par :

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Masque de sous-rÃ©seau

Le **masque** d'adresse IP termine quelle partie de l'adresse IP identifie le réseau et quelle partie identifie la machine.

```
IP : 192.168.1.10
Masque : 255.255.255.0 (/24)
RÃ©seau : 192.168.1.0
Machine : 0.0.0.10
Nombre de machines possibles : 254 (28 - 2)
```

```
graph LR
subgraph "DÃ©codage d'une adresse IPv4"
IP["192.168.1.10"]
MSK["Masque: 255.255.255.0<br/>( /24 )"]
NET["Partie rÃ©seau<br/>192.168.1"]
HOST["Partie machine<br/>.10"]
BROADCAST["Broadcast<br/>192.168.1.255"]
end
```

```
IP --> MSK
MSK --> NET
MSK --> HOST
NET --> BROAD
```

```
style IP fill:#3498db,color:#fff
style MSK fill:#e74c3c,color:#fff
style NET fill:#2ecc71,color:#fff
style HOST fill:#f39c12,color:#fff
style BROAD fill:#95a5a6,color:#fff
```

Commandes essentielles :

```
ip addr # Linux : voir sa configuration IP
ipconfig # Windows : voir sa configuration IP
ping 8.8.8.8 # Tester la connectivité Internet
tracert google.com # Visualiser le chemin des paquets
```

3. DNS – Domain Name System (15 min)

Le **DNS** est l'annuaire téléphonique d'Internet : il traduit les noms de domaine en adresses IP.

```
sequenceDiagram
participant U as Utilisateur
participant B as Navigateur (Client)
participant DNS as Serveur DNS
participant W as Serveur Web

U->>B: Tape "www.google.com"
```

```

B->>DNS: "Quelle est l'IP de www.google.com ?"
DNS->>DNS: Recherche dans sa base
DNS-->>B: "142.250.185.4"
B->>W: Connexion à 142.250.185.4:443 (HTTP GET)
W-->>B: Réponse HTML / JSON
B-->>U: Page affichée

```

Hiérarchie DNS :

1. **Racine (.)** : 13 serveurs racines dans le monde
2. **TLD (.com, .org, .dz, .fr)** : gérés par les registres
3. **Domaine (google.com, univ-mostaganem.dz)** : acheté par l'organisation
4. **Sous-domaine (www.google.com, moodle.univ-mostaganem.dz)**

Commande :

```

nslookup google.com # Interroger le DNS
dig google.com # Interrogation détaillée (Linux)

```

Application pédagogique : montrer aux élèves que `www.google.com` et `142.250.185.4` désignent le même serveur. Expliquer pourquoi les noms sont plus faciles à retenir que les adresses IP.

4. Modèle client-serveur (15 min)

Le **modèle client-serveur** est l'architecture fondamentale du Web : un client demande une ressource, un serveur la fournit.

```

graph LR
    subgraph "Modèle Client-Serveur"
        C1[Navigateur<br/>Client HTTP]
        C2[App mobile<br/>Client API]
        C3[Client FTP]

        S1[Serveur Web<br/>Apache / Nginx]
        S2[Serveur FTP]
        S3[Base de données]
    end

    C1 -->|"Requête HTTP GET"| S1
    C2 -->|"Requête API REST"| S1
    C3 -->|"Connexion FTP"| S2
    S1 -->|"Requête SQL"| S3

    S1 -->|"Réponse HTML/JSON"| C1
    S1 -->|"Réponse JSON"| C2
    S2 -->|"Fichiers"| C3

    style C1 fill:#3498db,color:#fff
    style C2 fill:#2ecc71,color:#fff
    style C3 fill:#f39c12,color:#fff
    style S1 fill:#e74c3c,color:#fff
    style S2 fill:#9b59b6,color:#fff
    style S3 fill:#95a5a6,color:#fff

```

Rôles : le **client** initie la requête (navigateur), le **serveur** fournit les ressources (machine toujours allumée).

Exemple : saisir `https://moodle.ens-mostaganem.dz` â†’ navigateur (client) interroge DNS â†’ envoie requête HTTP GET â†’ serveur répond avec HTML â†’ affichage.

Variantes : client-serveur classique, pair-à-pair (Torrent), 3-tiers (client â†’ app â†’ BDD).

5. Protocoles de communication (15 min)

Pile TCP/IP

```
graph TD
    subgraph "Pile TCP/IP (4 couches)"
        APP["Couche Application<br/>HTTP · HTTPS · FTP · SMTP · IMAP · POP3 · DNS"]
        TRANS["Couche Transport<br/>TCP (fiable) · UDP (rapide)"]
        INTER["Couche Internet<br/>IP · ICMP (ping)"]
        ACCES["Couche Accès<br/>Ethernet · Wi-Fi · ADSL"]
    end

    style APP fill:#e74c3c,color:#fff
    style TRANS fill:#3498db,color:#fff
    style INTER fill:#2ecc71,color:#fff
    style ACCES fill:#f39c12,color:#fff
```

Protocoles du Web

Protocole	Port	Usage	Chiffrement
HTTP	80	Navigation web (non sécurisée)	Non
HTTPS	443	Navigation web sécurisée (TLS/SSL)	Oui
FTP	21	Transfert de fichiers	Non
SFTP/FTPS	22/990	Transfert sécurisé	Oui

HTTPS ajoute une couche de chiffrement TLS entre le client et le serveur.

- Cadenas vert dans le navigateur
- Certificat numérique délivré par une autorité de certification (CA)
- Empêche l'espionnage des données sur le réseau

Protocoles de messagerie

- **SMTP** (Simple Mail Transfer Protocol) â€” port 25 ou 587 : envoi de courriels
- **IMAP** (Internet Message Access Protocol) â€” port 143 : réception et synchronisation multi-appareils (les messages restent sur le serveur)
- **POP3** (Post Office Protocol v3) â€” port 110 : réception et téléchargement local (les messages sont supprimés du serveur)

Comparaison IMAP vs POP3 :

IMAP	POP3
Messages sur le serveur	Messages téléchargés localement
Synchronisation multi-appareils	Un seul appareil
Accès depuis n'importe où	Accès hors ligne
Recommandé pour usage professionnel	Usage sur un seul ordinateur

Pour l'enseignant : utiliser **IMAP** (synchro multi-appareils), l'**email institutionnel** pour toute communication officielle, et rédiger des emails professionnels (objet clair, formule, signature).

6. Cloud computing (10 min)

Le **cloud computing** est la mise à disposition de ressources informatiques (stockage, calcul, applications) via Internet, sans gestion directe de l'infrastructure.

Modèles de service

Modèle	Exemple	Usage enseignant
SaaS (Software as a Service)	Moodle, Google Classroom, Office 365	Plateforme pédagogique
PaaS (Platform as a Service)	Heroku, PythonAnywhere	Déploiement d'applications
IaaS (Infrastructure as a Service)	AWS, Azure, OVH	Machines virtuelles

Services cloud pour l'enseignant

- **OneDrive / Google Drive** : stockage, partage et sauvegarde de documents pédagogiques
- **Moodle** : dépôt de cours, rendus de TD, forums, QCM en ligne
- **GitHub / GitLab** : hébergement de code, gestion de versions, projets NSI
- **Overleaf** : rédaction collaborative en LaTeX
- **Padlet / Framapad** : collaboration en temps réel

Avantages : accessibilité (depuis n'importe où), sauvegarde automatique, partage facilité, mise à jour automatique.

Inconvénients : dépendance à la connexion Internet, confidentialité des données (RGPD), coût potentiel.

QCM d'auto-évaluation

1. Que signifie LAN ?

a) Large Area Network b) Local Area Network c) Long Access Network

2. Quelle est la taille d'une adresse IPv4 ?

a) 16 bits b) 32 bits c) 128 bits

3. Quel est le rôle du DNS ?

a) Chiffrer les communications b) Traduire les noms de domaine en adresses IP c) Gérer les mots de passe

4. Quel protocole garantit une connexion Web chiffrée ?

a) HTTP b) HTTPS c) FTP

5. Quel protocole de messagerie est recommandé pour une synchronisation multi-appareils ?

a) POP3 b) SMTP c) IMAP

Conclusion (5 min)

Internet repose sur une infrastructure hiérarchique (routeurs, serveurs DNS) et des protocoles standardisés (TCP/IP, HTTP, SMTP). Le modèle client-serveur organise les échanges. La sécurité (HTTPS) et le cloud transforment les usages pédagogiques. En tant que futur enseignant, maîtriser ces bases vous permet d'utiliser les outils numériques avec discernement.

Prochain cours : Sécurité informatique et Éthique numérique.

Note pédagogique ENS

Transposition didactique :

- **SNT Seconde** (thème « Internet ») : utiliser `ping` et `tracert` pour visualiser le chemin des données. Activité débranchée : IPv4-routeurs avec des tables de routage.
- **NSI Première** : modéliser client-serveur avec des sockets en Python. Architecture TCP/IP vue en détail.
- **EMC** : débattre des enjeux du cloud (données personnelles, souveraineté numérique).

Conseil pour l'enseignant formateur : Faire simuler une requête DNS : un élève est le client, un autre le DNS, un autre le serveur web. Ils échangent des enveloppes avec les adresses IP.

Prolongement : Les étudiants peuvent explorer les couches applicatives (HTTP/2, DNS, DHCP, SMTP) et simuler un dialogue client-serveur avec des sockets Python. Proposer une activité complémentaire : créer un petit serveur web en Python avec Flask et le déployer localement pour comprendre le fonctionnement d'une application web.

Bibliographie

- Kurose & Ross « *Computer Networking: A Top-Down Approach* (8th ed.)
- Tanenbaum & Wetherall « *Computer Networks* (6th ed.)
- ANSSI « *Guide de sécurisation des réseaux* (disponible sur www.ssi.gouv.fr)
- RFC 1034 « *Domain Names – Concepts and Facilities*
- RFC 2616 « *HTTP/1.1*
- Documentation Mozilla : *How the Internet works* (developer.mozilla.org)

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_08.pdf

Cours Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 08 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques spÃ©cifiques

Ã€ l'issue de cette sÃ©ance, l'Ã©tudiant est capable de :

- Expliquer la triade CID (ConfidentialitÃ©, IntÃ©gritÃ©, DisponibilitÃ©)
- CrÃ©er un mot de passe robuste et comprendre le principe du MFA
- Identifier les signes d'une tentative de phishing
- Appliquer la rÃ¨gle de sauvegarde 3-2-1
- ConnaÃ®tre les principes du RGPD applicables en milieu scolaire
- Adopter une Ã©thique numÃ©rique responsable (cyberharcÃ©lement, droit d'auteur, IA gÃ©nÃ©rative)

Contexte rÃ©glementaire algÃ©rien

Ce cours s'inscrit dans le cadre rÃ©glementaire algÃ©rien de formation des enseignants :

- **ArrÃªtÃ© ministÃ©riel portant crÃ©ation des ENS (cycle Licence/Master)** â€” dÃ©finit le cadre institutionnel des Ã©coles Normales SupÃ©rieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algÃ©riennes** â€” fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **ArrÃªtÃ© 712 fixant le programme des concours ENS** â€” dÃ©termine les Ã©preuves et les programmes des concours d'accÃ©s aux ENS, en cohÃ©rence avec les contenus enseignÃ©s.
- **RÃ©fÃ©rentiel des compÃ©tences numÃ©riques algÃ©rien** â€” cadre national (en cours d'Ã©laboration) dÃ©finissant les compÃ©tences numÃ©riques attendues des enseignants.
- **Benchmark des 10 universitÃ©s algÃ©riennes** â€” Ã©tude comparative des pratiques d'enseignement de l'informatique, consultÃ©e pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Triade CID (ConfidentialitÃ©, IntÃ©gritÃ©, DisponibilitÃ©) â€” 15 min
2. Mots de passe robustes et authentification multi-facteurs â€” 15 min
3. Menaces numÃ©riques (phishing, malware, ransomware) â€” 15 min
4. RÃ¨gle de sauvegarde 3-2-1 â€” 10 min
5. RGPD et protection des donnÃ©es des Ã©lÃ©ves â€” 15 min
6. Ã©thique numÃ©rique pour l'enseignant â€” 15 min

7. QCM et conclusion â€” 5 min

1. Triade CID (15 min)

La **s curit  informatique** repose sur trois piliers fondamentaux, souvent repr sent s par le triangle CIA (Confidentiality, Integrity, Availability).

```
graph TD
    CIA((S CURIT   
INFORMATIQUE))

    CIA --> CONF[Confidentialit   
Donn es accessibles aux seuls  
utilisateurs  
autoris s]
    CIA --> INT[Int grit   
Donn es non modifi es  
de fa on non autoris e]
    CIA --> DISPO[Disponibilit   
Donn es et services  
accessibles en temps voulu]

    CONF <-->|D pendance| INT
    INT <-->|D pendance| DISPO
    DISPO <-->|D pendance| CONF

    style CIA fill:#e74c3c,color:#fff
    style CONF fill:#3498db,color:#fff
    style INT fill:#2ecc71,color:#fff
    style DISPO fill:#f39c12,color:#fff
```

Confidentialit  : garantir que seules les personnes autoris es acc dent   l'information.

M�canisme	Exemple concret
Chiffrement	HTTPS, disque dur chiffr� (BitLocker)
Contr�le d'acc�s	Mot de passe, badge RFID
Authentification	MFA, biom�trie

Int grit  : garantir que les donn es n'ont pas  t  alt r es.

M�canisme	Exemple concret
Somme de contr�le (hash)	MD5, SHA-256 (t�l�chargements)
Signature num�rique	Email sign�, certificats SSL
Journalisation	Audit trail, historique des modifications

Disponibilit  : garantir l'acc s aux donn es et services quand n cessaire.

M�canisme	Exemple concret
Redondance	Serveurs miroirs, RAID
Sauvegarde	R�gle 3-2-1
Plan de reprise	PRA (Plan de Reprise d'Activit�)

Exemple p dagogique : un carnet de notes en ligne. La **confidentialit ** garantit que seuls l'enseignant et l'administration y acc dent. L'**int grit ** emp che un  l ve de modifier sa note. La **disponibilit ** assure qu'on peut y acc der le jour du conseil de classe.

2. Mots de passe et authentification (15 min)

Caract ristiques d'un mot de passe robuste

Critère	Faible
Longueur	6-8 caractères
Complexité	minuscules uniquement
Dictionnaire	<code>password</code> , <code>123456</code> , <code>azerty</code>
Personnel	Date naissance, prénom

Exemples :

- soleil2025 → FAIBLE (mot du dictionnaire + année simple)
- ENS.M0st@g4nem_2026! → ROBUSTE (12 car., mélange de types, caractères spéciaux)

Authentification Multi-Facteurs (MFA)

Combinaison de plusieurs **facteurs** d'authentification :

```
FACTEUR 1 : Ce que je sais → Mot de passe, code PIN
+
FACTEUR 2 : Ce que j'ai → SMS, application (Google Auth), clé USB
+
FACTEUR 3 : Ce que je suis → Empreinte digitale, reconnaissance faciale
```

MFA en milieu scolaire : activer la double authentification sur Moodle, l'email institutionnel, et les services académiques.

Gestionnaire de mots de passe

- **Principe** : un seul mot de passe maître pour déverrouiller un coffre-fort numérique
- **Outils** : Bitwarden (gratuit, open source), KeePass, Apple Passwords
- **Avantage** : génération de mots de passe uniques et complexes pour chaque service

3. Menaces numériques (15 min)

Phishing (hameçonnage)

Technique d'ingénierie sociale où l'attaquant se fait passer pour une entité légitime afin de voler des identifiants.

```
graph TD
    subgraph "Cycle d'une attaque phishing"
        E["Expéditeur  
support@g00gle.com"] --> D["Destinataire  
Envoi massif des victimes"]
        D --> C["Objet : 'Compte bloqué'"]
        C --> S["Lien : http://fake.com"]
        S --> F["Saisie des identifiants sur page piégée"]
        F --> S
    end

    style E fill:#e74c3c,color:#fff
    style D fill:#f39c12,color:#fff
    style C fill:#3498db,color:#fff
    style S fill:#9b59b6,color:#fff
    style F fill:#e74c3c,color:#fff
```

Signes d'alerte :

1. Adresse expéditeur suspecte (ex: support@g00gle.com avec deux zéros)
2. Fautes d'orthographe et de grammaire

3. Message urgent (Â« votre compte sera fermÃ© sous 24h Â»)
4. Lien frauduleux (survoler le lien sans cliquer pour voir l'URL rÃ©elle)
5. PiÃ©ce jointe inattendue (.zip, .exe, .docm)

Malware (logiciel malveillant)

Type	Description	Propagation
Virus	S'attache Ã un fichier lÃ©gitime	PiÃ©ces jointes, tÃ©lÃ©chargements
Ver (worm)	Se propage seul sur le rÃ©seau	Email, vulnÃ©rabilitÃ©s rÃ©seau
Cheval de Troie	Se fait passer pour un logiciel lÃ©gitime	TÃ©lÃ©chargement piÃ©gÃ©
Spyware	Espionne l'activitÃ© de l'utilisateur	Sites web, logiciels gratuits
Ransomware	Chiffre les fichiers et demande ranÃ§on	Phishing, clÃ©s USB

Ransomware en dÃ©tail :

1. Infection via piÃ©ce jointe malveillante ou clÃ© USB
2. Chiffrement des fichiers (documents, photos, bases de donnÃ©es)
3. Affichage d'une demande de ranÃ§on (souvent en Bitcoin)
4. La victime perd ses donnÃ©es si elle ne paie pas (et mÃªme en payant)

Protection antivirale : Windows Defender (intÃ©grÃ© Ã Windows), solutions libres (ClamAV), audits rÃ©guliers.

4. RÃ©gle de sauvegarde 3-2-1 (10 min)

La **rÃ©gle 3-2-1** est la stratÃ©gie minimale de sauvegarde recommandÃ©e par les experts en sÃ©curitÃ©.

```
graph LR
    subgraph "StratÃ©gie de sauvegarde 3-2-1"
        ORIG["Copie originale<br/>PC / Serveur"]
        LOCALE["Copie locale<br/>Disque externe / NAS"]
        DISTANTE["Copie distante<br/>Cloud / Site diffÃ©rent"]
    end

    ORIG -->|"2. Sauvegarde locale"| LOCALE
    ORIG -->|"3. Sauvegarde distante"| DISTANTE

    NB["3 copies Ã 2 supports diffÃ©rents Ã 1 copie hors site"]

    style ORIG fill:#e74c3c,color:#fff
    style LOCALE fill:#3498db,color:#fff
    style DISTANTE fill:#2ecc71,color:#fff
    style NB fill:#f39c12,color:#fff,stroke:#333,stroke-dasharray: 5 5
```

Application concrÃ©te pour l'enseignant :

```
DonnÃ©es originales (PC portable)
â",
â"œâ"œâ"œ Copie 1 : Disque dur externe USB (mÃªme lieu)
â",
â""â"â"â" Copie 2 : Cloud institutionnel OneDrive (hors site)
```

Planification de sauvegarde :

- **Quotidienne** : documents de cours modifiés dans la journée
- **Hebdomadaire** : sauvegarde complète des dossiers pédagogiques
- **Mensuelle** : sauvegarde archivée (début et fin de mois)

Pièges à éviter :

- Ne pas tester ses sauvegardes (une sauvegarde non testée n'existe pas)
- Ne garder qu'une seule copie (au même endroit)
- Stocker les données sur une clé USB non chiffrée

5. RGPD et protection des données (15 min)

RGPD (Règlement Général sur la Protection des Données) est le Règlement européen 2016/679, applicable depuis mai 2018.

Principes fondamentaux

Principe	Application en milieu scolaire
Minimisation	Ne collecter que les données strictement nécessaires (nom, classe, pas de données médicales)
Transparence	Informers les élèves et leurs parents de l'utilisation des données
Consentement	Recueillir l'autorisation pour les photos, les publications
Droit d'accès	L'élève ou ses parents peuvent demander à voir leurs données
Droit à l'effacement	Supprimer les données quand elles ne sont plus utiles (fin d'année)
Sécurité	Protéger les listes d'élèves, notes et appréciations

Données particulièrement sensibles à protéger

Liste des élèves (nom, prénom, date de naissance)
 Notes, appréciations, bulletins
 Adresse, téléphone des parents
 Photos des élèves
 Aménagements spécifiques (PPS, PAP, notifications médicales)

Bonnes pratiques pour l'enseignant

- Utiliser exclusivement les **outils institutionnels** (Moodle, email universitaire, OneDrive académique)
- **Ne pas stocker** les données des élèves sur des services personnels (Google Drive perso, Dropbox)
- **Anonymiser** les données dans les publications et recherches
- **Durée de conservation** limitée à l'année scolaire en cours
- **Chiffrer** les fichiers sensibles (Word/Excel avec mot de passe, disque dur chiffré)

Référence : CNIL est le Guide de la protection des données dans l'éducation nationale

6. Éthique numérique pour l'enseignant (15 min)

Cyberharcèlement

- Formes : insultes, rumeurs, usurpation d'identité, diffusion de photos intimes

- **Statistiques** : 1 jeune sur 5 déclare avoir été victime (source : UNICEF)
- **Rôle de l'enseignant** : détecter, signaler (3018 en France, plateforme cybersignalement), accompagner
- **Prévention** : séances EMC, charte numérique de l'établissement

Droit d'auteur et licences

- **Images** : utiliser des banques libres de droit (Unsplash, Pixabay, Wikimedia Commons)
- **Licences Creative Commons (CC)** : CC0 (domaine public), CC BY (attribution obligatoire), CC BY-NC (pas d'usage commercial)
- **Citation des sources** : obligatoire dans tout document pédagogique
- **Textes** : domaine public (loi 70 ans après la mort de l'auteur) ou accord de l'éditeur

Intelligence Artificielle générative

Enjeux pour l'enseignement (2024-2026) :

Opportunité	Risque
Assistant de rédaction de cours	Plagiat par les élèves
Génération d'exercices personnalisés	Contournement des évaluations
Traduction automatique des consignes	Hallucinations et inexactitudes
Aide à la correction	Biais et discrimination

Recommandations :

- **Définir une politique** explicite sur l'usage de l'IA dans la classe
- **Former les élèves** à l'usage critique de l'IA
- **Adapter les évaluations** : plus d'oral, d'analyse, de production en classe
- **Citer systématiquement** quand l'IA est utilisée pour produire un document

Empreinte numérique

- Tout ce qui est publié en ligne laisse une trace (posts, photos, commentaires)
- **Conseil pour les futurs enseignants** : paramétrer la confidentialité des comptes personnels, séparer vie professionnelle et personnelle, ne rien publier qui pourrait nuire à votre image professionnelle

QCM d'auto-évaluation

1. Que signifie le « C » dans la triade CID ?

a) Connexion b) Confidentialité c) Certification

2. Quelle est la longueur minimale recommandée pour un mot de passe robuste ?

a) 8 caractères b) 12 caractères c) 6 caractères

3. Qu'est-ce qu'un ransomware ?

a) Un logiciel antivirus b) Un logiciel qui chiffre les fichiers et demande une rançon c) Un pare-feu

4. Quelle est la règle 3-2-1 ?

a) 3 mots de passe, 2 emails, 1 navigateur b) 3 copies, 2 supports, 1 copie hors site c) 3 utilisateurs, 2 serveurs, 1 sauvegarde

5. Selon le RGPD, les données des élèves doivent être :

a) Conservées indéfiniment b) Supprimées après l'année scolaire c) Publiées en ligne

Conclusion (5 min)

La sécurité informatique est l'affaire de tous. Connaître la triade CID, créer des mots de passe robustes, reconnaître un phishing, appliquer la règle 3-2-1 et respecter le RGPD sont des compétences essentielles pour tout enseignant du XXI^e siècle. L'éthique numérique (cyberharcèlement, droit d'auteur, IA) doit être intégrée dans votre pratique pédagogique quotidienne.

Prochain cours : Initiation à Python pour l'enseignement.

Note pédagogique ENS

Transposition didactique :

- **SNT Seconde** (thème « Données personnelles ») : analyser les paramètres de confidentialité des réseaux sociaux, créer une charte numérique de classe
- **NSI Première** : chiffrement symétrique/asymétrique, signature numérique, certificats SSL
- **EMC** : débat sur le cyberharcèlement, la liberté d'expression en ligne, les deepfakes

Activités recommandées :

1. Analyser collectivement 3 emails (1 légitime, 1 phishing évident, 1 phishing sophistiqué)
2. Simuler une attaque ransomware sur un dossier factice
3. Calculer le temps nécessaire pour casser un mot de passe de 6 vs 12 caractères
4. Rédiger une charte RGPD simplifiée pour la classe

Conseil pour l'enseignant formateur : Aborder la cybersécurité avec des exemples concrets tirés de la vie scolaire (mot de passe Pronote, phishing sur la messagerie académique, données personnelles sur les réseaux sociaux). Insister sur le rôle de l'enseignant comme premier rempart contre les cybermenaces en milieu scolaire. Utiliser des serious games comme « CyberEngage » ou « HackApp » pour sensibiliser sans anxiété.

Prolongement : Les étudiants peuvent explorer la cryptographie moderne (RSA, AES) et la messagerie chiffrée (Signal, ProtonMail). Proposer une activité complémentaire : implémenter un chiffrement par décalage (César) en Python, puis un chiffrement par substitution, et enfin discuter des limites de ces méthodes face à l'analyse fréquentielle.

Bibliographie

- Stallings & Brown « Computer Security: Principles and Practice (4th ed.)
- ANSSI « Guide de sélection des algorithmes de chiffrement (ssi.gouv.fr)
- CNIL « Protéger les données personnelles des élèves (cnil.fr)
- RGPD « Règlement (UE) 2016/679 du Parlement européen (eur-lex.europa.eu)

- Hadopi / Arcom " Guide du droit d'auteur pour les enseignants
- MIT " Cybersecurity for Everyone (cours en ligne)

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1ère_année_FR_09.pdf

Cours Informatique à l'ENS 1^{ère} année S^éance 09 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'1^{ère} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Justifier le choix de Python pour l'enseignement de la programmation
- Installer et configurer un environnement Python (IDLE, VS Code, Thonny)
- Définir et utiliser les types de base (int, float, str, bool)
- Structurer un programme avec conditions (if/elif/else) et boucles (for/while)
- Définir et appeler des fonctions avec paramètres et valeur de retour
- Lire et écrire dans un fichier texte depuis un script Python

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'issue duquel le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l'issue duquel fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** à l'issue duquel détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** à l'issue duquel cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** à l'issue duquel étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Pourquoi Python pour l'enseignement à l'1^{ère} année 10 min
2. Environnements de développement à l'1^{ère} année 10 min
3. Variables et types à l'1^{ère} année 15 min
4. Conditions (if/elif/else) à l'1^{ère} année 15 min
5. Boucles (for, while) à l'1^{ère} année 15 min
6. Fonctions (def, return, paramètres) à l'1^{ère} année 15 min

7. Entrées/sorties et fichiers â€ 10 min

8. QCM et conclusion â€ 5 min

1. Pourquoi Python pour l'enseignement ? (10 min)

Python est devenu le langage de programmation le plus utilis  dans l' ducation et la recherche.

```
graph LR
    subgraph "Pseudo-code algorithmique"
        PC["SI note >= 10<br/> AFFICHER 'Valid '<br/>SINON<br/> AFFICHER 'Non valid '"]
    end
    subgraph "Code Python"
        PY["if note >= 10:<br/> print('Valid ')<br/>else:<br/> print('Non valid ')"]
    end
    subgraph "Ex cution"
        EX["Entr e: 15<br/>Sortie: Valid "]
    end
    PC -->|"Traduction naturelle"| PY
    PY -->|"Interpr tation"| EX
    style PC fill:#f39c12,color:#fff
    style PY fill:#3498db,color:#fff
    style EX fill:#2ecc71,color:#fff
```

Atouts p dagogiques : syntaxe lisible (proche du pseudo-code), typage dynamique, gratuit/open source, biblioth ques riches, communaut   ducative active. Python est le langage officiel du programme NSI en France.

Utilisations en contexte scolaire :

- **Math matiques** : calculs, simulation, trac s (matplotlib)
- **Langues** : analyse de textes, quiz de vocabulaire
- **Sciences** : traitement de donn es, graphiques
- **SNT/NSI** : programme officiel â€ langage de r f rence

2. Environnements de d veloppement (10 min)

Outil	Type	Public cible	Avantage
IDLE	IDE int�gr�	D�butants	Install� avec Python
Thonny	IDE �ducatif	Coll�ge/Lyc�e	D�bogueur visuel, id�al pour d�couvrir
VS Code	�diteur pro	Lyc�e/Universit�	Extensions, terminal int�gr�
Replit	En ligne	Tous	Aucune installation
Google Colab	Notebook Web	Recherche/NSI	Tutoriels partag�s

Recommandation : Thonny pour d buter, puis VS Code pour les projets.

```
# V rifier Python
python --version
# Ex cuter un script
echo 'print("Bonjour ENS !")' > hello.py && python hello.py
```

3. Variables et types (15 min)

En Python, les variables sont des étiquettes pointant vers des objets. Pas de déclaration de type explicite.

```
age = 20 # int
pi = 3.14159 # float
nom = "Madani" # str
est_enseignant = True # bool

print(type(age)) # <class 'int'>
print(f"{nom} a {age} ans") # f-string (Python 3.6+)

# Conversion de type
age_str = "20"
age_int = int(age_str) # "20" à 20
note = float(input("Note : ")) # Saisie à float
```

Parallèle pseudo-code à Python :

Concept	Pseudo-code	Python
Affectation	<code>age ← 20</code>	<code>age = 20</code>
Affichage	<code>AFFICHER age</code>	<code>print(age)</code>
Saisie	<code>LIRE age</code>	<code>input()</code>
Commentaire	<code>// texte</code>	<code># texte</code>
Comparaison	<code>age == 18</code>	<code>age == 18</code>

Piège : = est l'affectation, == est la comparaison.

4. Conditions (15 min)

```
graph TD
    DEBUT(["DÉBUT"]) --> SAISIE[Saisir note]
    SAISIE --> COND{"note >= 10 ?"}
    COND -->|Oui| VALIDE[Afficher 'Validé']
    COND -->|Non| NONVALIDE[Afficher 'Non validé']
    VALIDE --> FIN(["Fin"])
    NONVALIDE --> FIN

    style DEBUT fill:#2ecc71,color:#fff
    style FIN fill:#e74c3c,color:#fff
    style COND fill:#f39c12,color:#fff

    note = float(input("Note sur 20 : "))
    if note >= 16:
        print("Très bien")
    elif note >= 14:
        print("Bien")
    elif note >= 12:
        print("Assez bien")
    elif note >= 10:
        print("Passable")
    else:
        print("Non validé")
```

Opérateurs : ==, !=, <, >, <=, >=, and, or, not

Règles d'or :

- Ne pas oublier les : après if, elif, else
- Indentation cohérente (4 espaces)

- `float(input())` pour les nombres d'écimaux

5. Boucles (15 min)

Boucle `for` pour le parcours d'une séquence

```
for i in range(5):
    print(f"Itération {i}")

for i in range(1, 10, 2):
    print(i) # 1, 3, 5, 7, 9

noms = ["Ali", "Sara", "Mohamed"]
for nom in noms:
    print(f"Bonjour {nom}")

for i, nom in enumerate(noms):
    print(f"{i+1}. {nom}")
```

Boucle `while` pour l'itération conditionnelle

```
reponse = ""
while reponse != "oui" and reponse != "non":
    reponse = input("Continue ? (oui/non) : ").lower()

compteur = 0
while compteur < 5:
    print(f"Compteur : {compteur}")
    compteur += 1

graph LR
    subgraph "Boucle for"
        F1["for i in range(5):"]
        F2["print(i)"]
        F3["i: 0 1 2 3 4"]
    end
    subgraph "Boucle while"
        W1["while condition:"]
        W2["instruction"]
        W3["Condition vérifiée<br/> chaque itération"]
    end
    F1 --> F2 --> F3
    W1 --> W2 --> W3 --> W1
    style F1 fill:#3498db,color:#fff
    style W1 fill:#e74c3c,color:#fff
```

Quand utiliser quoi ? `for` quand le nombre d'itérations est connu (parcours de séquence). `while` quand on attend une condition.

6. Fonctions (15 min)

```
def calculer_moyenne(notes):
    """Calcule la moyenne d'une liste de notes."""
    return sum(notes) / len(notes)

def saluer(nom, message="Bienvenue"):
    print(f"{message}, {nom} !")

notes = [15, 12, 18, 10, 14]
moy = calculer_moyenne(notes)
print(f"Moyenne : {moy:.2f}")
```

```
saluer("Ali") # Bienvenue, Ali !
saluer("Sara", "Bonjour") # Bonjour, Sara !

def notes_min_max(notes):
    return min(notes), max(notes) # Retourne un tuple

mini, maxi = notes_min_max(notes)
print(f"Min: {mini}, Max: {maxi}")
```

Pourquoi des fonctions ? Réutilisabilité, lisibilité, testabilité, modularité.

Bonnes pratiques : noms en verbes, docstring entre `"""`, un retour par fonction.

7. Entrées/sorties et fichiers (10 min)

```
# Saisie utilisateur
nom = input("Entrez votre nom : ")
age = int(input("Entrez votre Âge : "))

# Écriture fichier
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Nom,Note\n")
    f.write("Ali,15\n")
    f.write("Sara,18\n")
# Le with ferme automatiquement

# Lecture fichier
with open("notes.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())

# Modes : "r" lecture, "w" écriture (écrase), "a" ajout

# Traitement CSV simple
with open("notes.txt", "r") as f:
    f.readline() # Ignorer en-tête
    total = 0
    n = 0
    for ligne in f:
        nom, note = ligne.strip().split(",")
        total += float(note)
        n += 1
    print(f"Moyenne : {total/n:.2f}")
```

Contexte pédagogique : le format CSV est omniprésent dans l'éducation (export Moodle, Excel, bulletins).

8. Mini-projet (5 min)

Quiz de vocabulaire anglais/français combinant tous les concepts :

```
questions = {
    "chat": "cat", "chien": "dog", "maison": "house",
    "livre": "book", "école": "school", "professeur": "teacher"
}

def poser_question(mot_fr, mot_en):
    reponse = input(f"Traduire '{mot_fr}' : ").strip().lower()
    if reponse == mot_en:
        print("Correct !")
```

```

return True
print(f"â€- Faux. R ponse : {mot_en}")
return False

score = 0
for fr, en in questions.items():
    if poser_question(fr, en):
        score += 1
print(f"Score : {score}/{len(questions)}")

```

QCM d'auto- valuation

1. Quelle fonction affiche du texte dans la console en Python ?

a) `input()` b) `print()` c) `write()`

2. Quel type retourne `input()` ?

a) `int` b) `float` c) `str`

3. Quel mot-cl  d finit une fonction ?

a) `function` b) `def` c) `define`

4. Quelle boucle est id ale pour parcourir une liste ?

a) `while` b) `for` c) `repeat`

5. Quel mode d'ouverture permet l' criture ( crase) ?

a) `"r"` b) `"w"` c) `"a"`

Conclusion (5 min)

Python est id al pour l'enseignement : syntaxe claire, concepts universels (variables, conditions, boucles, fonctions), applications concr tes (traitement de fichiers, quiz). La manipulation de CSV pr pare les futurs enseignants   traiter les donn es p dagogiques.

Prochain cours : Projet Python   g n rateur de fiches de r vision et calculateur de moyenne.

Note p dagogique ENS

Transposition didactique :

Niveau	Th�me	Activit� recommand�e
SNT Seconde	Programmation	Utiliser Thonny pour les premiers pas. �crire un script qui affiche �« Bonjour »
NSI Premi�re	Types et fonctions	Impl�menter une fonction <code>moyenne(notes)</code> qui calcule la moyenne d'une liste de notes.
NSI Terminale	R�cursivit� et fichiers	Lire un fichier CSV de notes, calculer les statistiques, exporter un bulletin.
Math�matiques	Simulation	�crire une fonction qui simule le lancer d'un d� (random), r�p�ter N fois et calculer la moyenne.
Interdisciplinaire	Quiz de vocabulaire	Le mini-projet de quiz (section 8) peut �tre adapt� en cours de langue : l'�criture d'un quiz.

D marche p dagogique progressive :

1. **Lire** : analyser un script existant, rep rer les blocs (variables, conditions, boucles)

2. **Modifier** : changer des valeurs, ajouter une condition,  tendre une boucle

3. **Cr  er** :   crire un script r  pondant    un cahier des charges simple

4. **  valuer** : tester avec diff  rents jeux de donn  es, d  tecter et corriger les erreurs

Conseil pour l'enseignant formateur :

- Utiliser **Thonny** pour la d  couverte (d  bogueur pas    pas visible), puis **VS Code** pour les projets
- Privil  gier les **exercices contextuels** (calcul de moyenne, quiz, convertisseur) plut  t que des exercices d  contextualis  s
- Montrer **syst  matiquement** le parall  le pseudo-code    Python pour ancrer les concepts vus en cours d'algorithmique (s  ance 05)
- Pr  voir des **fiches d'aide** (memento Python) que les   tudiants conservent pour les s  ances suivantes
- **  viter** : utiliser Thonny portable ou Replit pour les premi  res s  ances (pr  f  rer une installation locale)

Prolongement : Les   tudiants peuvent explorer les biblioth  ques Python pour l'enseignement : `turtle` (graphiques), `pygame` (jeux), `matplotlib` (visualisation). Proposer une activit   compl  mentaire : cr  er un petit exercice de vocabulaire fran  ais avec des flashcards al  atoires utilisant `tkinter` ou un quiz interactif en ligne de commande avec `chronom  tre (time)`. Ces activit  s sont directement transposables en cours de langue ou d'histoire-g  ographie.

Bibliographie

- Downey   " *Think Python* (2nd ed., O'Reilly)
- Documentation Python 3 : docs.python.org
- *Python pour l'enseignement* (Eduscol)

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1  re_ann  e_FR_10.pdf

Cours Informatique à l'ENS 1^{ère} année S^éance 10 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'1^{ère} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Structurer un projet Python en modules (main.py, fonctions.py, donnees.csv)
- Gérer les erreurs d'exécution avec try/except
- Lire et écrire des fichiers CSV avec le module csv
- Développer un générateur de fiches de révision à partir d'un CSV
- Développer un calculateur de moyenne avec export CSV
- Présenter son projet selon la grille d'évaluation de la soutenance

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Rappel Python à l'aide du pseudo-code à Python à 10 min
2. Structure d'un projet Python à 10 min
3. Gestion d'erreurs (try/except) à 15 min
4. Module csv (lire/écrire) à 10 min
5. Projet : Générateur de fiches de révision à 20 min
6. Projet : Calculateur de moyenne avec export CSV à 15 min

7. Conseils soutenance et grille d'Ã©valuation â€” 10 min

1. Rappel Python (10 min)

Concept	Pseudo-code	Python
Variable	<code>age â† 20</code>	<code>age = 20</code>
Condition	<code>SI age >= 18</code>	<code>if age >= 18:</code>
Boucle pour	<code>POUR i DE 0 Ã 4</code>	<code>for i in range(5):</code>
Boucle tant que	<code>TANT QUE x > 0</code>	<code>while x > 0:</code>
Fonction	<code>FONCTION f(x):</code>	<code>def f(x):</code>
Retour	<code>RETOURNER x</code>	<code>return x</code>
Tableau	<code>t[1]</code> (indice 1)	<code>t[0]</code> (indice 0)
Affichage	<code>AFFICHER x</code>	<code>print(x)</code>
Saisie	<code>LIRE x</code>	<code>x = input()</code>

Attention : les listes Python commencent Ã l'indice 0, contrairement au pseudo-code qui commence souvent Ã 1.

2. Structure d'un projet Python (10 min)

```
graph TD
    ROOT[projet_pedagogique/] --> MAIN[main.py]
    ROOT --> FONC[fonctions.py]
    ROOT --> CSV[donnees.csv]
    MAIN -->|importe| FONC
    FONC -->|lit/Ã©crit| CSV
    style ROOT fill:#34495e,color:#fff
    style MAIN fill:#e74c3c,color:#fff
    style FONC fill:#3498db,color:#fff
    style CSV fill:#2ecc71,color:#fff

    projet_pedagogique/
    â€œâ€ main.py # Point d'entrÃ©e
    â€œâ€ fonctions.py # Fonctions mÃ©tier
    â€œâ€ donnees.csv # DonnÃ©es
    â€œâ€ README.txt # Documentation

    # fonctions.py
    def calculer_moyenne(notes): return sum(notes)/len(notes)

    def mention(m): return "TB" if m>=16 else "B" if m>=14 else "AB" if m>=12 else "P" if m>=10
    else "NV"
```

3. Gestion d'erreurs â€” try/except (15 min)

```
graph TD
    DEBUT([DÃ©but]) --> TRY["try: note = float(input())"]
    TRY --> VERIF["0 <= note <= 20 ?"]
    VERIF -->|Oui| OK[Afficher]
    VERIF -->|Non| RV["raise ValueError"]
    RV --> EXCEPT["except ValueError:"]
    TRY -. ->|erreur| EXCEPT
```

```

EXCEPT --> ERR[Message]
OK --> FIN; ERR --> FIN
style TRY fill:#3498db; style EXCEPT fill:#e74c3c

while True:
    try:
        note = float(input("Note (0-20) : "))
        if 0 <= note <= 20:
            break
        print("La note doit être entre 0 et 20.")
    except ValueError:
        print("Veuillez entrer un nombre.")

    try:
        with open("notes.csv", "r") as f:
            data = f.read()
    except FileNotFoundError:
        print("Fichier introuvable.")
    except PermissionError:
        print("Permission refusée.")

```

Règles : spécifier le type d'exception (ValueError, FileNotFoundError), proposer un message clair, ne jamais utiliser except: nu.

4. Module csv (10 min)

```

import csv

# Lecture (liste de listes)
with open("notes.csv", "r", encoding="utf-8") as f:
    lecteur = csv.reader(f)
    next(lecteur) # Ignorer l'entête
    for ligne in lecteur:
        nom, note1, note2 = ligne
        print(f"{nom}: {note1}, {note2}")

# Lecture (dictionnaires)
with open("notes.csv", "r", encoding="utf-8") as f:
    lecteur = csv.DictReader(f)
    for ligne in lecteur:
        print(ligne["nom"], ligne["note1"])

# Écriture
with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Nom", "Moyenne", "Mention"])
    writer.writerow(["Ali", 15.5, "Bien"])
    writer.writerow(["Sara", 18.0, "Très bien"])

# Écriture avec DictWriter
with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    colonnes = ["Nom", "Moyenne", "Mention"]
    writer = csv.DictWriter(f, fieldnames=colonnes)
    writer.writeheader()
    writer.writerow({"Nom": "Ali", "Moyenne": 15.5, "Mention": "Bien"})

```

Important : toujours newline="" en écriture (évitez les doubles interlignes sous Windows).

5. Projet : Générateur de fiches de révision (20 min)

Ã%tape 1 : revision.csv

```

Concept,DÃ©finition
Variable,Bloc qui stocke une valeur en mÃ©moire
Fonction,Bloc de code rÃ©utilisable
Boucle,RÃ©pÃ©te un bloc d'instructions
Condition,ExÃ©cute selon une expression boolÃ©enne
File,Structure LIFO (dernier ajoutÃ© = premier retirÃ©)
File,Structure FIFO (premier ajoutÃ© = premier retirÃ©)
DNS,Traduit les noms de domaine en adresses IP

```

Ã%tape 2-3 : fonctions.py + main.py

```

import csv, random

def charger_fiches(f):
    try:
        with open(f, encoding="utf-8") as fh:
            return list(csv.DictReader(fh))
    except FileNotFoundError:
        print(f"Fichier introuvable.")
    return []

def lancer_quiz(fs):
    if not fs: return
    random.shuffle(fs); score = 0
    for f in fs:
        print(f"\nConcept : {f['Concept']}")
        input("EntrÃ©e..."); print(f"DÃ©finition : {f['DÃ©finition']}")
        if input("Correct ? (o/n): ")=="o": score+=1
    print(f"Score: {score}/{len(fs)}")

if __name__ == "__main__":
    lancer_quiz(charger_fiches("revision.csv"))

```

6. Projet â€” Calculateur de moyenne (15 min)

```

import csv

def saisir_notes():
    eleves = []
    print("\n--- SAISIE DES NOTES ---")
    while True:
        nom = input("\nNom (ou 'fin') : ").strip()
        if nom.lower() == "fin": break
        notes = []
        for i in range(1, 4):
            while True:
                try:
                    n = float(input(f" Note {i} (0-20) : "))
                    if 0 <= n <= 20:
                        notes.append(n); break
                except ValueError:
                    print("Note invalide.")
            print("Entrez un nombre.")
        moy = round(sum(notes)/len(notes), 2)
        eleves.append({"nom": nom, "notes": notes, "moyenne": moy})
        print(f"â†’ {nom}: {moy:.2f}")
    return eleves

def stats(eleves):
    if not eleves: return
    moyennes = [e["moyenne"] for e in eleves]
    print(f"\nEffectif: {len(eleves)} | Classe: {sum(moyennes)/len(moyennes):.2f}")

```

```

print(f"Max: {max(moyennes):.2f} | Min: {min(moyennes):.2f}")

def exporter(elevs, fichier="resultats.csv"):
    with open(fichier, "w", newline="", encoding="utf-8") as f:
        cols = ["Nom", "Note1", "Note2", "Note3", "Moyenne", "Mention"]
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for e in elevs:
            m = e["moyenne"]
            mention = "TB" if m >= 16 else "B" if m >= 14 else "AB" if m >= 12 else "P" if m >= 10 else "NV"
            w.writerow({"Nom": e["nom"], "Note1": e["notes"][0],
                        "Note2": e["notes"][1], "Note3": e["notes"][2],
                        "Moyenne": f"{m:.2f}", "Mention": mention})
        print(f"ExportÃ© dans {fichier}")

if __name__ == "__main__":
    e = saisir_notes()
    if e: stats(e); exporter(e)

graph LR
    SPEC[SpÃ©cifications] --> CONCEP[Conception]
    CONCEP --> IMPL[ImplÃ©mentation]
    IMPL --> TEST[Test]
    TEST --> VALID{VALIDATION ?}
    VALID -->|Oui| SOUT[Soutenance]
    VALID -->|Non| IMPL
    SOUT --> FIN([Projet validÃ©])

style SPEC fill:#3498db,color:#fff
style CONCEP fill:#2ecc71,color:#fff
style IMPL fill:#e67e22,color:#fff
style TEST fill:#9b59b6,color:#fff
style VALID fill:#f1c40f,color:#333
style SOUT fill:#e74c3c,color:#fff
style FIN fill:#2c3e50,color:#fff

```

7. Conseils soutenance (10 min)

CritÃ©re	Poids	Attentes
FonctionnalitÃ©	30 %	ExÃ©cution sans erreur
Structure du code	20 %	Fichiers sÃ©parÃ©s, docstrings
Gestion d'erreurs	15 %	try/except
DonnÃ©es CSV	10 %	Import/export correct
README.txt	10 %	Objectif, utilisation
PrÃ©sentation orale	15 %	5 min de dÃ©mo + questions

Soutenance : prÃ©sentation (30s) â†’ dÃ©mo (2min30) â†’ code (1min) â†’ amÃ©liorations (30s) â†’ questions (30s).

Projet	CSV	FonctionnalitÃ©s
Quiz vocabulaire	mots.csv	Tirage alÃ©atoire, score
Tables multiplication	difficultes.csv	Exercices alÃ©atoires

Gestionnaire notes	eleves.csv	Moyennes, export bulletin
Flashcards	concepts.csv	Mode apprentissage

QCM d'auto-évaluation

1. Comment importer les fonctions d'un fichier `fonctions.py` ?

a) `include fonctions` b) `from fonctions import *` c) `using fonctions`

2. Quelle exception pour un fichier introuvable ?

a) `ValueError` b) `FileNotFoundError` c) `TypeError`

3. Quel paramètre éviter les doubles interlignes CSV sous Windows ?

a) `encoding="utf-8"` b) `newline=""` c) `mode="wb"`

4. Quelle méthode de `csv` retourne les lignes en dictionnaires ?

a) `csv.reader()` b) `csv.DictReader()` c) `csv.reader_dict()`

5. Pondération de la Fonctionnalité dans la grille ?

a) 20 % b) 30 % c) 15 %

Conclusion (5 min)

Ce dernier cours vous a donné les outils pour un projet Python complet : modules, gestion d'erreurs, CSV, et soutenance. Vous êtes capables de créer des outils pédagogiques concrets (quiz, générateurs d'exercices, calculateurs de notes) pour vos futures classes.

Félicitations !

Note pédagogique ENS

Transposition didactique : Le projet Python final valide l'ensemble des compétences du module (C1 à C8). La démarche suivie (spécifications → conception → implémentation → test → soutenance) est exactement celle du Bac NSI en France. Les compétences développées sont transposables au lycée comme suit :

Compétence	Activité transposable au lycée
Structures de projet	Le découpage en <code>main.py</code> / <code>fonctions.py</code>
Gestion d'erreurs	<code>try/except</code> avec messages explicites : exigible en NSI Terminale
Fichiers CSV	Le format CSV (export Moodle, Excel, Pronote) est le format d'échange de données le plus courant
Quiz pédagogique	Programme interactif utilisable dans toutes les disciplines : langues, histoire, sciences
Calculateur de moyenne	Automatisation du suivi des notes : outil concret que le futur enseignant peut déployer dans sa classe

Activités recommandées pour la classe :

1. **Analyse d'un projet existant** → Distribuer un projet Python complet et demander aux élèves d'identifier le rôle de chaque fichier et fonction

2. **Cahier des charges** → Faire rédiger le cahier des charges avant de coder (compétence « Expression des besoins » du CRCN)

3. **Revue de code** – En binôme, chaque étudiant révise le code de l'autre et propose des améliorations
4. **Présentation orale** – La soutenance de 5 minutes pr  pare aux   preuves orales du Bac (Grand Oral, NSI)
5. **Adaptation    un niveau inf  rieur** – Demander aux   tudiants de simplifier le quiz pour des   l  ves de 6e (activit   de transposition didactique)

Conseil pour l'enseignant formateur : Le projet final est l'aboutissement du module ; il doit   tre valoris   comme une production personnelle. Accompagner les   tudiants dans le choix du sujet (quiz p  dagogique, calculateur de moyenne, jeu   ducatif) et veiller    ce qu'ils respectent les   tapes de d  veloppement (sp  cifications    conception    impl  mentation    test). La soutenance de 5 minutes doit   tre chronom  tr  e et   valu  e selon des crit  res clairs (qualit   du code, pertinence p  dagogique, pr  sentation orale).

Prolongement : En ENS 2  me ann  e, le projet peut   tre enrichi par la POO (classes `Question`, `Quiz`, `Joueur`), le d  veloppement Web avec Flask (formulaires et base de donn  es SQLite), l'exposition des r  sultats via une API REST, ou encore un projet interdisciplinaire (quiz d'histoire-g  ographie, exercices de math  matiques, flashcards de langues).

Bibliographie

- Documentation Python 3 : docs.python.org/fr/3/tutorial/
- Module csv : docs.python.org/fr/3/library/csv.html
- Downey – *Think Python* (O'Reilly)
- Programme NSI – eduscol.education.fr